

17. Bölüm

Tasarım Desenleri

17.1 Desen Nedir?

Nesne yönelimli programlamanın ortaya çıkışıyla beraber, daha önceden yazılmış hazır bileşenlerin (**component**) yeniden kullanımı (**reusing**) konusunda oldukça önemli ilerlemeler sağlamıştır. Bunun paralelinde tecrübelerin (**experience**) yeniden kullanımı konusunda da çeşitli çalışmalar yapılmış ve **desen** (**pattern**) kavramı ortaya çıkmıştır. Desenlerin aksine **anti-desen** (**anti-pattern**) ise **başlangıçta çekici görünen** ancak uygulama sürecinde **verimsiz olduğu kanıtlanmış yaygın hatalı çözümlerdir**.

Yazılım geliştirme öncesi, geliştirme aşması ve sonrasında daha önce yaşanmış çeşitli problemlere karşı birçok kimse tarafından çeşitli çözümler getirilmiştir. **Desenler, daha önce geliştirme yapan programcıların yanlış yaparak doğru yapmayı öğrendikleri başarılı tecrübelerin anlatılması için bir araçtır**. Gerçekleştirilen bu çözümlerin, daha sonradan yaşanacak benzer nitelikli problemlerde kullanılması amacıyla desenler kullanılır. Desen kavramının temelleri Mimar *Christopher Alexander*'ın 1970 sonlarında başlattığı çalışmalara dayanmaktadır¹.

1987 yılında uluslararası “*Nesne Yönelimli Programlama, Sistemler, Diller ve Uygulamalar*” konferansına kadar desenlerle ilgili bir çalışma ortaya çıkmamıştır². Bu tarihten sonra ise başta *Grady Booch*, *Richard Helm*, *Erich Gamma* ve *Kent Beck* olmak üzere pek çok bilim adamı desenlerle ilgili makale ve sunumlar yayınlamışlardır. 1994 yılında *Erich Gamma*, *Richard Helm*, *Ralph Johnson* ve *John Vlissides* tarafından yayınlanan “*Tasarım Desenleri: Tekrar kullanılabilir Nesne Yönelimli Yazılımın Temelleri*” kitabı, tasarım desenlerin yazılımda kullanılmasında dönüm noktası olmuştur³. Yazılımlarda kullanılan desenler yedi ana başlıkta toplanırlar. Bunlar;

1. Mimari desenler (**architectural pattern**)
2. Analiz desenleri (**analysis pattern**)
3. Tasarım desenleri (**design pattern**)
4. Kodlama desenleri (**implementation pattern**)
5. Test desenleri (**test pattern**)
6. Çözüm desenleri (**solution pattern**)
7. Veri desenleri (**data pattern**)

Bu desenlerin en önemlisi ve ilk gelişenlerden birisi, **tasarım desenleridir** (**design pattern**). Tasarım desenleri aslında, Nesne Yönelimli Tasarım Prensiplerini, yazılımlara uygulamanın bir başka ifadesidir. Bu nedenle çok önemlidir. Tasarım desenleri ise üç ana başlıkta toplanmaktadır.

1. Nesne imali ile ilgili desenler (**creational patterns**)
2. Yapısal desenler (**structural patterns**)
3. Davranışlarla ilgili desenler (**behavioral patterns**)

Desenlerin çok sık kullanıldığı programlama yöntemine günümüzde **desen yönelimli programlama** (**pattern oriented programming**) adı verilmektedir⁴. İzleyen sayfalarda verilen desen UML diyagramlarının birçoğu *Erich Gamma*'nın **Design Pattern CD** yayınından alınmıştır⁵. Bölümün devamında verilecek tasarım desenlerinin UML diyagramları visual-paradigm adresinden alınmıştır⁶.

17.2 Nesne İmalatı ile İlgili Desenler

Nesne imalatı ile ilgili desenler (**creational patterns**), nesne (**object**) örnekleme ile ilgili ortaya çıkabilecek çeşitli problemlere çözüm getirmektedirler. Bu desenlerde nesneler dinamik olarak imal edilirler, çünkü değişim yönetiminde statik nesneler kullanılmazlar.

¹<http://gee.cs.oswego.edu/dl/ca/ca/ca.html>

²OOPSLA, Object Oriented Programming, Systems, Languages, and Applications

³Design Patterns: Elements of Reusable Object-Oriented Software, ISBN 0-201-63361-2

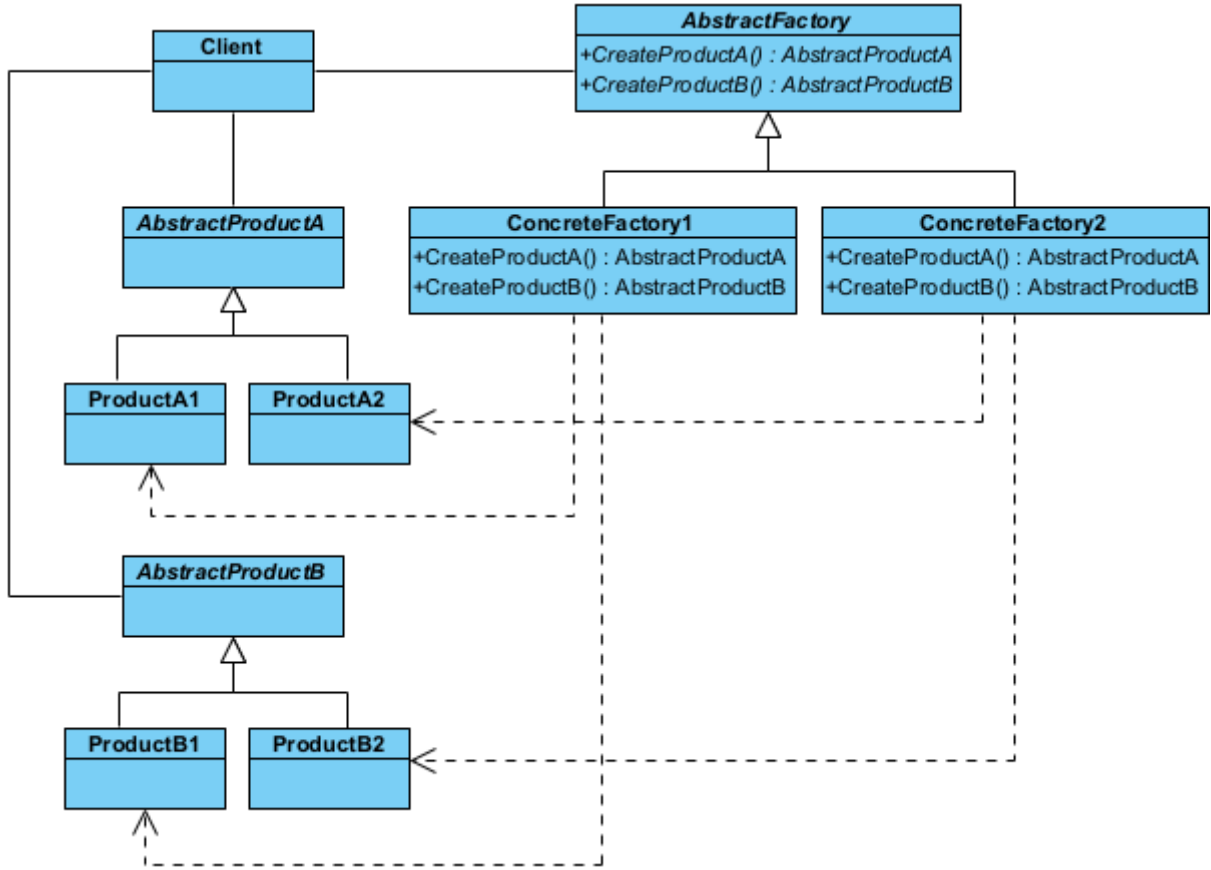
⁴www.mathcs.sjsu.edu/faculty/pearce/patterns.html

⁵Design Pattern CD, Addison-Wesley, 1998

⁶<https://circle.visual-paradigm.com/category/uml-diagrams/gof-design/>

§17.2.1 Soyut Fabrika

Soyut fabrika deseninde (**abstract factory pattern**) amaç, nesnelerin imal edilmesi ve kullanılması ile ilgili olan kısımlarının birbirinden ayrılmasıdır. Bu amaca, birbiriyle ilgili ya da birbirine bağımlı olabilecek nesnelerin imal edilmesinde, nesnelerin somut (**concrete**) sınıflarını değil soyut (**abstract**) sınıfları kullanılarak ulaşılır. Somut sınıflar istemciden izole edilir ve programcıya somut sınıfları daha kolay değiştirme için olanak sağlar.



Şekil 17.1: Soyut Fabrika Deseni UML Diyagramı

Bu tasarım deseninde, birbirleriyle ilişkili veya bağımlı nesne ailelerini, somut sınıflarını belirtmeden oluşturmak için kullanılan imalatla ilgili (**creational**) bir desendir. "Fabrikaların fabrikası" olarak da adlandırılır. Eğer sisteminizin farklı ürün aileleriyle (örneğin; Windows uyumlu buton/pencere ailesi ve Mac uyumlu buton/pencere ailesi) çalışması gerekiyorsa ve bu ailelerin birbirine karışmasını istemiyorsanız bu deseni kullanırsınız.

- ◊ Soyut Fabrika Arayüzü (**AbstractFactory**), ilgili tüm ürünleri oluşturacak yöntemlerin imzalarını (şablonunu) tanımlar. "Bir buton oluştur" ve "Bir pencere oluştur" gibi soyut görevleri belirler.
- ◊ Somut Fabrikalar (**ConcreteFactory**), belirli bir ürün ailesine ait nesneleri fiilen oluşturur. Örneğin **ModernMobilyaFabrikasi** sadece modern tarzdaki sandalyeleri ve masaları üretir.
- ◊ Soyut Ürünler (**AbstractProduct**), ürün ailesindeki her bir temel ürün türü için bir arayüz tanımlar. Tüm sandalyelerin sahip olması gereken ortak özellikleri (Örnek olarak **otur()**) belirler.
- ◊ Somut Ürünler (**ConcreteProduct**), soyut ürünlerin farklı varyasyonlarını (ailelerini) temsil eden gerçek sınıflardır. **ModernSandalye** veya **AntikaSandalye** gibi sınıflar burada yer alır.

Bu deseni kodlayalım;

```

//C++ 14 ve üzeri ile derlenmeli
#include <iostream>
#include <string>
#include <memory> // unique_ptr için gerekli
#include <vector>

//1. Products (Ürünler)
--- Abstract Products ---
class AbstractProductA {

```

```

public:
    AbstractProductA(std::string pAdi) : productName(pAdi) {}
    virtual ~AbstractProductA() = default; // Sanal yıkıcı (Virtual Destructor) çok önemli!
    → unique_ptr'nin alt sınıfları doğru silmesi için gereklidir.
    virtual std::string getUrunAdi() const { return productName; }
private:
    std::string productName;
};

class ConcreteProductA1 : public AbstractProductA {
public:
    using AbstractProductA::AbstractProductA; //Temel sınıfın TÜM yapıcılarını içeri aktarır!
};

class ConcreteProductA2 : public AbstractProductA {
public:
    using AbstractProductA::AbstractProductA; //Temel sınıfın TÜM yapıcılarını içeri aktarır!
};

// --- Abstract Product B ---
class AbstractProductB {
public:
    AbstractProductB(std::string pAdi) : productName(pAdi) {}
    virtual ~AbstractProductB() = default;
    virtual std::string getUrunAdi() const { return productName; }
    // Etkileşim için raw pointer (AbstractProductA*) kullanabiliriz. Çünkü sahiplik (ownership)
    → almıyoruz, sadece gözlemliyoruz.
    void interact(const AbstractProductA* a) const {
        std::cout << this->productName << " ürünü " << a->getUrunAdi() << " ile etkileşim
        → halinde..." << std::endl;
    }
private:
    std::string productName;
};

class ConcreteProductB1 : public AbstractProductB {
public:
    using AbstractProductB::AbstractProductB; //Temel sınıfın TÜM yapıcılarını içeri aktarır!
};

class ConcreteProductB2 : public AbstractProductB {
public:
    using AbstractProductB::AbstractProductB; //Temel sınıfın TÜM yapıcılarını içeri aktarır!
};

//2. Factory (Fabrika)
--- Abstract Factory ---
class AbstractFactory {
public:
    virtual ~AbstractFactory() = default;
    // unique_ptr döndürerek nesnenin sahipliğini çağırana devrediyoruz
    virtual std::unique_ptr<AbstractProductA> createProductA() = 0;
    virtual std::unique_ptr<AbstractProductB> createProductB() = 0;
};

class ConcreteFactory1 : public AbstractFactory {
public:
    std::unique_ptr<AbstractProductA> createProductA() override {
        return std::make_unique<ConcreteProductA1>("A1 ürünü");
    }
    std::unique_ptr<AbstractProductB> createProductB() override {
        return std::make_unique<ConcreteProductB1>("B1 ürünü");
    }
};

class ConcreteFactory2 : public AbstractFactory {
public:

```

```

std::unique_ptr<AbstractProductA> createProductA() override {
    return std::make_unique<ConcreteProductA2>("A2 ürünü");
}
std::unique_ptr<AbstractProductB> createProductB() override {
    return std::make_unique<ConcreteProductB2>("B2 ürünü");
}
};

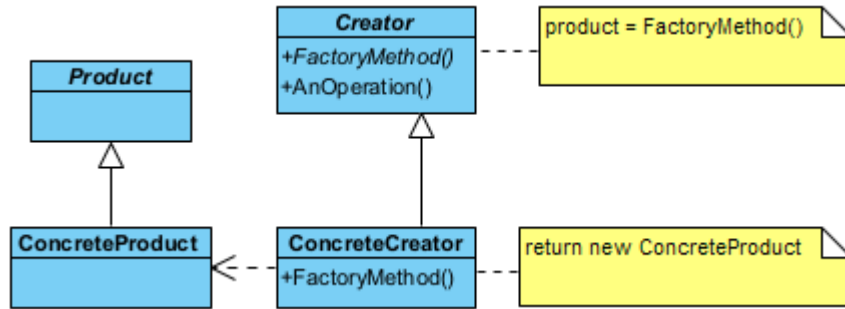
// 3. Client (İstemci)
int main() {
    // Fabrikaları unique_ptr ile oluşturuyoruz
    auto factory1 = std::make_unique<ConcreteFactory1>();
    auto factory2 = std::make_unique<ConcreteFactory2>();
    // Ürünleri oluşturuyoruz - Bellek yönetimi artık otomatik!
    auto productA = factory1->createProductA();
    std::cout << "Fabrika 1 ürünü: " << productA->getUrunAdi() << std::endl;
    auto productB = factory2->createProductB();
    std::cout << "Fabrika 2 ürünü: " << productB->getUrunAdi() << std::endl;
    // .get() ile akıllı işaretçinin içindeki ham adresi geçici olarak kullanabiliriz
    productB->interact(productA.get());
    // delete yazmamıza gerek yok; main biterken her şey otomatik temizlenir.
}

/*Program Çıktısı:
Fabrika 1 ürünü: A1 ürünü
Fabrika 2 ürünü: B2 ürünü
B2 ürünü ürünü A1 ürünü ile etkileşim halinde...
*/

```

§17.2.2 Fabrika Yöntemi

Fabrika yöntemi desende (**factory method pattern**) amaç, imal edilecek nesnenin sınıfını kesin olarak belirtmeden nesne yaratma işleminin gerçekleştirilmesidir. Bu desenin görevi, nesne üretim işini alt sınıflara



Şekil 17.2: Fabrika Yöntemi Deseni UML Diyagramı

birakır. Hangi nesnenin üretileceğine çalışma zamanında karar verilir.

- ♦ Üretici (**Creator**), fabrika yöntemini tanımlayan soyut sınıftır.
- ♦ Ürün (**Product**), üretilecek nesnelerin ortak arayüzüdür.
- ♦ Somut Üretici (**ConcreteCreator**), ihtiyaca göre "A ürünü" veya "B ürünü" üreten gerçek fabrikadır.

Bu deseni kodlayalım;

```

//C++ 14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <memory> // unique_ptr için şart

//1. Products (Ürünler)
// --- Ürün Sınıfları ---
class Product {
public:
    Product(string pAdi) : productName(pAdi) {}
    virtual ~Product() = default; // Alt sınıfların güvenli silinmesi için sanal yıkıcı
    virtual string getUrunAdi() const {

```

```

        return productName;
    }
private:
    string productName;
};

class ConcreteProduct : public Product {
public:
    using Product::Product; // Üst sınıfın yapıcısını miras al
};

//2. Creators (İmalatçılar)
// --- Creator (İmalatçı) Sınıfları ---
class Creator {
public:
    virtual ~Creator() = default;
    virtual unique_ptr<Product> factoryMethod() = 0; // Sahipliği devreden Fabrika Yöntemi
    virtual void anOperation() = 0;
};

class ConcreteCreator : public Creator {
public:
    unique_ptr<Product> factoryMethod() override {
        // C++14 kullanıyorsan make_unique, C++11 ise unique_ptr<T>(new T())
        return make_unique<ConcreteProduct>("Fabrika Yöntemi Ürünü");
    }
    void anOperation() override {
        cout << "Fabrika Yöntemini Kullanan Sınıf Başka İşlemler de yapabilir..." << endl;
    }
};

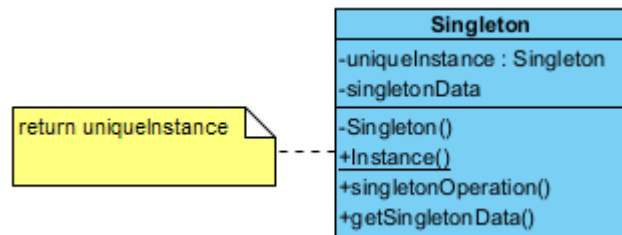
//3. Client (İstemci)
int main() {
    // 1. Creator'ı akıllı gösterici ile oluştur
    unique_ptr<Creator> imalatci = make_unique<ConcreteCreator>();
    // 2. Ürünü oluştur (Sahiplik imalatçıdan 'urun' değişkenine geçer)
    unique_ptr<Product> urun = imalatci->factoryMethod();
    cout << "İmalatçı tarafından imal edilen ürün: " << urun->getUrunAdi() << endl;
    imalatci->anOperation();
    // Manuel 'delete' yapmaya GEREK YOK. main fonksiyonu bittiğinde urun ve imalatci otomatik
    // olarak silinir.
}

/*Program Çıktısı:
İmalatçı tarafından imal edilen ürün: Fabrika Yöntemi Ürünü
Fabrika Yöntemini Kullanan Sınıf Başka İşlemler de yapabilir...
*/

```

§17.2.3 Tekil Nesne

Tekil Nesne deseninde (**singleton pattern**) amaç, bir sınıftan yalnızca bir örnek (**instance**) imal etmektir. Bir sınıfın yalnızca bir nesnesinin olmasına izin verilir, birden fazla olmasına izin verilmez.



Şekil 17.3: Tekil Deseni UML Diyagramı

Modern C++'ta (C++11 ve sonrası) bu desen, "Meyers' Singleton" olarak bilinen çok daha zarif, güvenli ve performanslı bir yapıya dönüşmüştür. Şimdi Singleton sınıfını kodlayalım;

```
1 //C++11 ve üzeri ile derlenmelidir
```

```

2  #include <iostream>
3
4  //1. Singletone (Tekil Nesne) Sınıfı
5  class Singleton {
6  private:
7      // Yapıcı yöntem gizli
8      Singleton() : singletonData(10) {
9          std::cout << "Nesne ilk kez oluşturuldu.\n";
10     }
11     int singletonData;
12 public:
13     // Kopyalama ve atama engellenir
14     Singleton(const Singleton&) = delete;
15     Singleton& operator=(const Singleton&) = delete;
16     // En modern erişim noktası
17     static Singleton& Instance() {
18         // Statik yerel değişken: Sadece ilk çağrıda oluşturulur,
19         // program sonlanırken otomatik silinir. Thread-safe'dir.
20         static Singleton instance;
21         return instance;
22     }
23     void anOperation() const {
24         std::cout << "Modern tekil nesne görev basında...\n";
25     }
26     int getSingletonData() const { return singletonData; }
27 };
28
29 //2. Client (İstemci)
30 int main() {
31     // Nesneye referans üzerinden erişmek daha güvenlidir
32     Singleton& s1 = Singleton::Instance();
33     Singleton& s2 = Singleton::Instance();
34
35     if (&s1 == &s2) {
36         std::cout << "s1 ve s2 AYNI nesnedir.\n";
37     }
38     s1.anOperation();
39 }
40 /*Program Çıktısı:
41 Nesne ilk kez oluşturuldu.
42 s1 ve s2 AYNI nesnedir.
43 Modern tekil nesne görev basında...
44 */

```

Yukarıda kodlaması yapılan bu desenin amacı, bir sınıftan (**Singleton**) uygulama ömrü boyunca sadece bir adet nesne üretilmesini garanti etmektir.

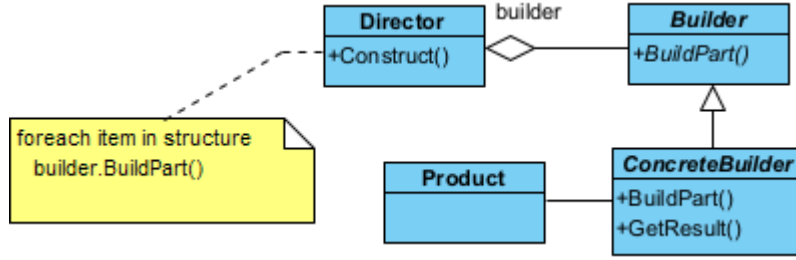
- ◊ Mahrem Yapıcı (**private constructor**), dışarıdan rastgele nesne üretilmesini engeller. 7. Satırda tanımlanmıştır.
- ◊ Statik Tekil Örnek (**static unique instance**), oluşturulan tek nesneyi kendi içinde saklar. 19. Satırda tanımlanmıştır.
- ◊ Statik Erişim Yöntemi (**static access method**), nesneye her yerden erişim sağlayan kapıdır. 16. Satırda tanımlanmıştır.

§17.2.4 Kurucu

Kurucu deseninde (**builder pattern**) amaç, çok karmaşık bir nesnenin imal edilmesiyle ilgili işlemleri bir başka sınıfa bırakmaktır. Böylece nesnenin gösterimi ve kullanılması ilişkin kodlar ile imalatına ilişkin kodlar birbirinden ayrılmış olur.

Bu deseni bir fast-food restoranındaki "Menü Hazırlama" sürecine benzetebilirsiniz. Hamburgerin içine turşu konulup konulmayacağı, içeceğin boyutu veya yan ürünün ne olacağı gibi adımlar tek tek belirlenir, ancak en sonunda ortaya tek bir "Menü" nesnesi çıkar.

- ◊ Ürün (**Product**), inşa edilmeye çalışılan karmaşık nesnedir. Genellikle çok sayıda parçadan veya kon-



Şekil 17.4: Kurucu Deseni UML Diyagramı

figürasyondan oluşur.

- ◊ Soyut Kurucu (**Builder**), ürünün parçalarını oluşturmak için gerekli olan tüm adımları (yöntemleri) arayüz olarak tanımlar. "Ekmek koy", "Köfte ekle", "Sos dök" gibi soyut inşa adımlarını belirler.
- ◊ Somut Kurucu (**ConcreteBuilder**), soyut kurucudaki adımları fiilen gerçekleştirir ve ürünün bir örneğini saklar. "Vejetaryen Burger" veya "Double Burger" gibi farklı ürün varyasyonlarını nasıl inşa edeceğini bilir.
- ◊ Yönetici (**Director**), inşa adımlarının hangi sırayla çağrılacağını kontrol eder. İstemciden (**Client**) aldığı kurucu nesnesini kullanarak, doğru sırayla (Örn: Önce temel, sonra yan ürünler) inşa sürecini yürütür.

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <vector>
#include <string>
#include <memory> // std::unique_ptr için
#include <utility> // std::move için

//1. Ürün Sınıfı (Product)
class Product {
public:
    explicit Product(std::string name) : productName(std::move(name)) {}
    void addPart(const std::string& part) {
        parts.push_back(part);
    }
    void showParts() const {
        std::cout << productName << " Ürününün parçaları:\n";
        // Modern C++: Range-based for loop ve const kullanımı
        for (const auto& part : parts) {
            std::cout << "- " << part << "\n";
        }
    }
private:
    std::string productName;
    std::vector<std::string> parts;
};

//2. Kurucu (Builder) Sınıfları
//Kurucu Arayüzü (Abstract Builder)
class Builder {
public:
    virtual ~Builder() = default; // Sanal yıkıcı şarttır
    virtual void buildPartKisim1() = 0;
    virtual void buildPartKisim2() = 0;
    virtual void buildPartKisim3() = 0;
    // Ürünün mülkiyetini (ownership) dışarıya devreden yöntem
    virtual std::unique_ptr<Product> getResult() = 0;
};

//Somut Kurucu (Concrete Builder)
class ConcreteBuilder : public Builder {
public:
    ConcreteBuilder() {
        // make_unique ile güvenli oluşturma
    }

```



```

        product = std::make_unique<Product>("Modern C++ Kitabı");
    }
    void buildPartKisim1() override { product->addPart("Bolum 1: Akilli Gösterişçiler"); }
    void buildPartKisim2() override { product->addPart("Bolum 2: Lamda Ifadeleri"); }
    void buildPartKisim3() override { product->addPart("Bolum 3: Tasarım Desenleri"); }
    std::unique_ptr<Product> getResult() override {
        // Sahipliği dışarıya (Client'a) taşıyoruz. Builder artık buna sahip değil.
        return std::move(product);
    }
private:
    std::unique_ptr<Product> product;
};

//3. Yönetici (Director)
class Director {
public:
    // Builder'ı referans olarak almak sahiplik karmaşasını önler
    void construct(Builder& builder) {
        builder.buildPartKisim1();
        builder.buildPartKisim2();
        builder.buildPartKisim3();
    }
};

//4. Client (İstemci)
int main() {
    // Nesneleri stack üzerinde veya unique_ptr ile oluşturuyoruz
    Director director;
    auto builder = std::make_unique<ConcreteBuilder>();
    // İnşa süreci
    director.construct(*builder);
    // Ürünü teslim al (Sahiplik builder'dan main'e geçti)
    std::unique_ptr<Product> product = builder->getResult();
    if (product) {
        product->showParts();
    }
    // Manuel 'delete' yok! Kapsam bitince her şey otomatik temizlenir.
}
/*Program Çıktısı:
Modern C++ Kitabı Urununun parcalari:
- Bolum 1: Akilli Isaretciler
- Bolum 2: Lambda Ifadeleri
- Bolum 3: Tasarim Desenleri
*/

```

§17.2.5 Prototip

Prototip deseni (**prototype pattern**), mevcut bir nesnenin kopyasını (klonunu) oluşturmak için kullanılan imalatla ilgili bir desendir. Bu desenin temel amacı, bir nesneyi oluşturmanın maliyeti (zaman, bellek veya işlem gücü açısından) çok yüksek olduğunda, sıfırdan **new** işleci ile üretmek yerine mevcut bir örneği klonlayarak yeni nesneler elde etmektir.

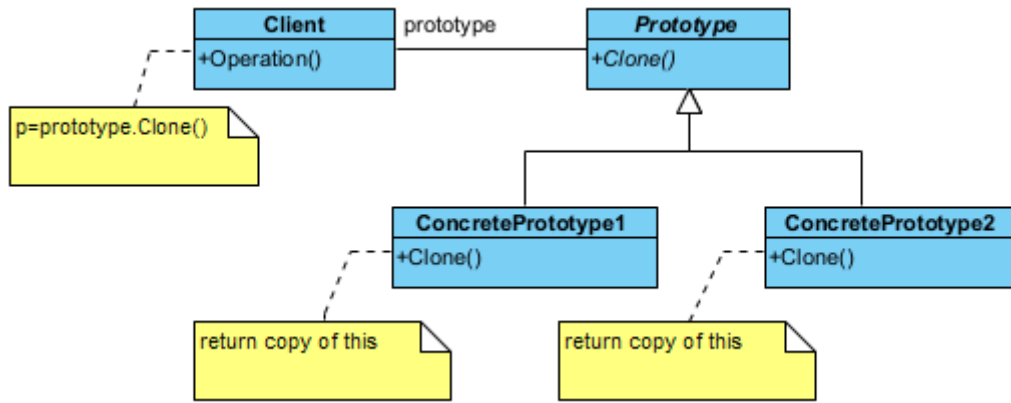
- ◊ Soyut Prototip (**Prototype**), nesnenin kendisini kopyalayabilmesi için gerekli olan arayüzü tanımlar. Genellikle sadece bir **clone()** yönteminin imzasını barındırır.
- ◊ Somut Prototip (**ConcretePrototype**), soyut prototip arayüzünü uygular ve asıl kopyalama işlemini gerçekleştirir. Kendi verilerini yeni bir nesneye aktarır. C++ dünyasında bu işlem genellikle Kopya yapıcı (**copy constructor**) kullanılarak yapılır.
- ◊ İstemci (**Client**), yeni bir nesneye ihtiyaç duyduğunda, sınıfa gidip "yeni bir tane üret" demek yerine, elindeki prototip nesnesine "kendini klonla" der.

Şimdi bu deseni kodlayalım;

```

#include <iostream>
#include <memory>
#include <string>

```

Şekil 17.5: Prototip Deseni UML Diyagramı

```

// 1. Prototip (Prototype) Sınıfları
//--- Abstract Prototype ---
class Prototype {
protected: // Miras alan sınıfların erişebilmesi için protected
    std::string data_;
public:
    virtual ~Prototype() = default;
    // Nesneyi kopyalayıp akıllı işaretçi olarak döndürür
    virtual std::unique_ptr<Prototype> clone() const = 0;
    void showData() const {
        std::cout << "Nesne Verisi: " << data_ << std::endl;
    }
};

// --- Concrete Prototype ---
class ConcretePrototype : public Prototype {
public:
    // Yapıcı fonksiyon (Constructor)
    ConcretePrototype(std::string data) {
        data_ = std::move(data);
    }
    // Kopyalama yapıcı (Copy Constructor) clone() için gerekli
    std::unique_ptr<Prototype> clone() const override {
        // Mevcut nesnenin (*this) kopyasını oluşturur
        return std::make_unique<ConcretePrototype>(*this);
    }
};

// 2. Client (İstemci)
void operation() {
    // 1. Orijinal nesneleri unique_ptr ile oluşturalım (Güvenli yol)
    std::unique_ptr<Prototype> object1 = std::make_unique<ConcretePrototype>("40.30");
    object1->showData();
    // 2. Klonlama işlemi
    // clone() bir unique_ptr döndürür, bu yüzden alıcı da unique_ptr olmalı
    std::unique_ptr<Prototype> clone1 = object1->clone();
    clone1->showData();
    std::unique_ptr<Prototype> object2 = std::make_unique<ConcretePrototype>("Mehmet");
    object2->showData();
    std::unique_ptr<Prototype> clone2 = object2->clone();
    clone2->showData();
    // Akıllı göstericiler (unique_ptr) sayesinde manuel 'delete' yapmaya gerek kalmaz!
}

int main() {
    operation();
}

```

```
/*Program Çıktısı:
Nesne Verisi: 40.30
Nesne Verisi: 40.30
Nesne Verisi: Mehmet
Nesne Verisi: Mehmet
*/
```

Nesneleri kopyalama söz konusu olduğunda, genel olarak üç tür kavramdan bahsedilir;

Üstünkörü Kopyalama

Üstünkörü kopyalamada (shallow copy); C++'ta bir sınıf için özel bir kopyalama yapıcısı (**copy constructor**) yazmazsanız, derleyici varsayılan olarak bir "memberwise copy" (üye düzeyinde kopyalama) yapar. Eğer sınıfta ham işaretçiler (**raw pointers**) varsa, bu işlem teknik olarak bir bit düzeyi kopyalamadır (**bitwise copy**). Eğer nesnenin dinamik bellekten (new ile) yer ayırdıysa, hem orijinal nesne hem de kopya nesne aynı bellek adresini göstermesi bir sorundur. Nesnelerden biri yok edildiğinde (yıkıcı çalıştığında) belleği serbest bırakır. Diğerisi hala o adresi kullanmaya çalıştığında sarkant gösterici (**dangling pointer**) veya çift serbest bırakma (**double free**) hatasıyla karşılaşırız. Bu, C++ geliştiricilerinin en meşhur kabuslarından biridir.

Derinlemesine Kopyalama

Derinlemesine kopyalamada (deep copy); Prototip deseninin gerçek ruhunu yansıtan yöntemdir. Bu yöntemde, nesnenin sahip olduğu tüm kaynaklar (işaret edilen bellek alanları dahil) yeni nesne için baştan tahsis edilir. Prototip deseninde genellikle **virtual T* clone() const;** şeklinde bir yöntem tanımlanır. Bu yöntem içerisinde **new** anahtar kelimesiyle nesnenin tam bir kopyası oluşturulur. Orijinal nesne ile kopya nesne tamamen bağımsızdır. Birinin verisini değiştirmek veya birini yok etmek diğerini etkilemez.

Tembel Kopyalama

(Lazy Copy / Copy-on-Write) **Tembel kopyalamada (lazy copy);** Modern C++ dünyasında (özellikle eski **std::string** gerçekleştirmelerinde ve bazı büyük kütüphanelerde) performans optimizasyonu için kullanılır. Nesne kopyalandığında başlangıçta üstünkörü kopyalama yapılır ve bir **referans sayacı (reference counter)** tutulur. İki nesne de aynı veriyi sadece okuyorsa (**read-only**), fazladan bellek harcamaya gerek yoktur. Nesnelerden biri veriyi değiştirmeye (write) kalktığı anda, gerçek bir derinlemesine kopyalama yapılır ve kopyalar birbirinden ayrılır. Buna **Copy-on-Write (COW)** denir. Modern C++'ta **std::shared_ptr** ve **std::weak_ptr** kullanımı, bu mantığın bir kısmını (kaynak paylaşımı) otomatikleştirir.

Prototip deseninde **clone()** yöntemini yazarken, sınıfınızın içerisinde **std::vector** veya **std::string** gibi standart kütüphane nesneleri kullanıyorsanız, C++ bunları varsayılan olarak derinlemesine kopyalama (kendi içerindeki kopyalama mantığı sayesinde) ile kopyalar. Ancak ham gösterici (**int***, **char***) kullanıyorsanız, derin kopyalamayı manuel olarak yönetmek gerekir.

17.3 Yapısal Desenler

Yapısal desenler (structural pattern), sınıflar ve nesnelerin çok olduğu daha karmaşık programlarda getirilen yapısal çözüm yöntemleridir.

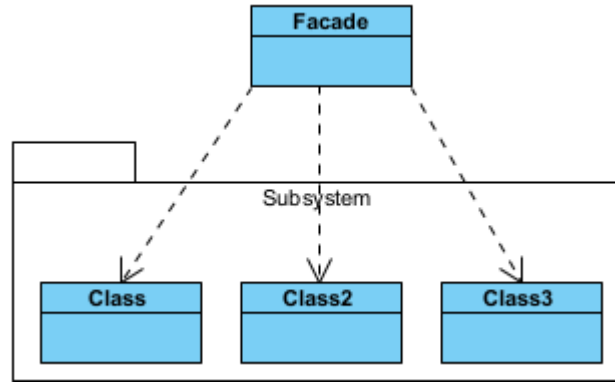
§17.3.1 Önyüz

Önyüz deseninde (facade pattern) karmaşık bir alt sistemdeki (**subsystem**) çok sayıda sınıfı ve arayüzü, kullanımı kolay tek bir üst düzey arayüz arkasında gizlemek için kullanılan yapısal bir desendir. Bu deseni bir restoranın "Garsonu" gibi düşünebilirsin. Sen sadece sipariş verirsin; garson ise mutfak, bulaşıkhanenin ve kasanın karmaşıklığıyla senin adına ilgilenir.

- ♦ Önyüz (**Facade**), karmaşık alt sistemleri bilen ve onları yöneten "ana giriş kapısı"dır. İstemciden (**Client**) gelen basit isteği, alt sistemdeki doğru nesnelere yönlendirir ve onları uygun sırayla çalıştırır.
- ♦ Alt Sistem Sınıfları (**SubSystem**), işin asıl detaylarını yapan çok sayıda sınıftır (Örn: SesSistemi, Isik-Denetleyici, Projeksiyon). **Facade** nesnesinden tamamen habersizdirler. Kendi işlerini yaparlar. **Facade** sadece bunları koordine eder.
- ♦ İstemci (**Client**), sistemi kullanan son kullanıcı veya başka bir kod parçasıdır. Alt sistemin karmaşık detaylarıyla (hangi sınıf hangi sırayla çağrılacak gibi) uğraşmaz; sadece **Facade**'in sunduğu basit yöntemleri çağırır.

Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir.
#include <iostream>
#include <memory> // std::unique_ptr için
```



Şekil 17.6: Önyüz Deseni UML Diyagramı

```

//1. Alt Sistemler (Subsystems)
// Bu sınıflar karmaşık iş mantığını barındırır.
class SubSystem1 {
public:
    void operation11() { std::cout << "Subsystem1: Yöntem 1 çalıştırıldı." << std::endl; }
};
class SubSystem2 {
public:
    void operation22() { std::cout << "Subsystem2: Yöntem 2 çalıştırıldı." << std::endl; }
};
class SubSystem3 {
public:
    void operation31() { std::cout << "Subsystem3: Yöntem 1 çalıştırıldı." << std::endl; }
    void operation32() { std::cout << "Subsystem3: Yöntem 2 çalıştırıldı." << std::endl; }
    void operation33() { std::cout << "Subsystem3: Yöntem 3 çalıştırıldı." << std::endl; }
};

//2. Facade (Önyüz) Sınıfı
//İstemciye (Client) basit bir arayüz sunar, karmaşayı yönetir.
class Facade {
private:
    // Akıllı göstericiler ile otomatik bellek yönetimi
    std::unique_ptr<SubSystem1> subsystem1;
    std::unique_ptr<SubSystem2> subsystem2;
    std::unique_ptr<SubSystem3> subsystem3;
public:
    Facade() : subsystem1(std::make_unique<SubSystem1>()),
        subsystem2(std::make_unique<SubSystem2>()), subsystem3(std::make_unique<SubSystem3>()) {
        // Yapıcıda üye ilklendirme listesi (member initializer list) kullanımı daha
        // performanslıdır.
    }

    // Yıkıcı (Destructor) yazmaya gerek yok, unique_ptr otomatik temizler.
    void executeComplexOperation() {
        std::cout << "--- Facade Karma sık İşlemi Başlatıyor ---" << std::endl;
        subsystem1->operation11();
        subsystem2->operation22();
        subsystem3->operation31();
        subsystem3->operation33();
        subsystem3->operation32();
        std::cout << "--- İşlem Tamamlandı ---" << std::endl;
    }
};

//3. Client (İstemci)
int main() {

```

```

    // Client tarafında Facade nesnesini doğrudan Stack üzerinde oluşturuyoruz.
    Facade facade;
    facade.executeComplexOperation();
}
/*Program Çıktısı:
--- Facade Karma sik Islemi Baslatiyor ---
Subsystem1: Yöntem 1 calistirildi.
Subsystem2: Yöntem 2 calistirildi.
Subsystem3: Yöntem 1 calistirildi.
Subsystem3: Yöntem 3 calistirildi.
Subsystem3: Yöntem 2 calistirildi.
--- Islem Tamamlandi ---
*/

```

Aşağıda bu desene gerçek dünya örneği olarak alt sistemleri olan bir bilgisayarın başlatılması modellenmiştir;

```

//C++17 ile derlenmelidir.
#include <iostream>
#include <string>
#include <string_view> // C++17: Bellek kopyalamayı azaltan hafif metin görünümü
#include <iomanip>      // Çıktı biçimlendirme (hex) için

// --- Sabit Tanımlamalar ---
namespace BootConstants {
    constexpr long ADDRESS = 0x7C00;
    constexpr int SECTOR = 0;
    constexpr int SECTOR_SIZE = 512;
}

//1. --- Alt Sistemler (Subsystems) ---
class CPU {
public:
    void freeze() const {
        std::cout << "CPU: Durduruldu (Freeze)." << std::endl;
    }
    void jump(long position) const {
        std::cout << "CPU: Adrese atlandı: 0x" << std::hex << std::uppercase << position <<
            << std::endl;
    }
    void execute() const {
        std::cout << "CPU: Komutlar calistiriliyor..." << std::dec << std::endl;
    }
};

class Memory {
public:
    // const char* yerine string_view kullanarak daha güvenli veri aktarımı sağlıyoruz
    void load(long position, std::string_view data) const {
        std::cout << "Memory: 0x" << std::hex << position
            << " adresine veri yuklendi: [" << data << "]" << std::endl;
    }
};

class HardDrive {
public:
    std::string_view read(long lba, int size) const {
        std::cout << "HardDrive: Sector " << std::dec << lba << " uzerinden "
            << size << " byte veri okundu." << std::endl;
        return "OS_KERNEL_DATA";
    }
};

//2. --- Facade (Görünüş / Önyüz) ---

```

```

class ComputerFacade {
private:
    // Alt sistemleri üyeler olarak tutuyoruz.
    // Eğer bu alt sistemler çok ağırsa unique_ptr kullanılabilir.
    CPU cpu_;
    Memory memory_;
    HardDrive hardDrive_;
public:
    void start() {
        std::cout << "Bilgisayar baslatiliyor...\n" << std::string(35, '-') << std::endl;
        // Facade, karmaşık adımları bir orkestra şefi gibi yönetir:
        cpu_.freeze();
        // Diski oku ve belleğe aktar
        auto data = hardDrive_.read(BootConstants::SECTOR, BootConstants::SECTOR_SIZE);
        memory_.load(BootConstants::ADDRESS, data);
        // İşlemciyi yeni adrese yönlendir ve çalıştır
        cpu_.jump(BootConstants::ADDRESS);
        cpu_.execute();
        std::cout << std::string(35, '-') << "\nSistem hazır!" << std::endl;
    }
};

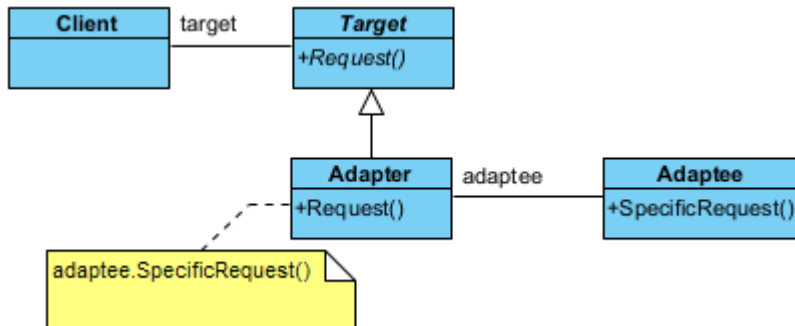
//3. --- Client (İstemci) ---
int main() {
    // İstemci (Kullanıcı) sadece "başlat" komutunu bilir.
    ComputerFacade computer;
    computer.start();
}

/*Program Çıktısı:
Bilgisayar baslatiliyor...
-----
CPU: Durduruldu (Freeze).
HardDrive: Sector 0 üzerinden 512 byte veri okundu.
Memory: 0x7c00 adresine veri yuklendi: [OS_KERNEL_DATA]
CPU: Adrese atlandı: 0x7C00
CPU: Komutlar çalıştırılıyor...
-----
Sistem hazır!
*/

```

§17.3.2 Adaptör

Adaptör deseni (**adapter pattern**), yabancı bir arayüzü istemcinin anlayacağı bir arayüze dönüştürür. Yani farklı arayüzlere sahip sınıfların, iletişim ve etkileşim kurabilecekleri ortak bir nesne oluşturarak birlikte çalışmalarına izin verir. Bu desenin amacı, uyumsuz iki arayüzün (**interface**) birlikte çalışmasını sağlar. Burada



Şekil 17.7: Adaptör Deseni UML Diyagramı

yabancı arayüze yaptırılacak işin bir şekilde yapıldığı ve zaten yapılan bu işlemin istemcinin (**Client**) isteğine uygun hale getirildiği düşünülmelidir. Bir nevi elektrik prizi dönüştürücüsü görevi görür.

- ◊ İstemci (**Client**), mevcut sistemle çalışan ana kod.
- ◊ Uyumsuz Nesne (**Adaptee**), sisteme dahil edilmek istenen ama arayüzü farklı olan nesne.

- ◊ Hedef (**Target**), istemcinin (**Client**) beklediği arayüz.
- ◊ Uyarlayıcı (**Adapter**), uyumsuz nesneyi sarmalayarak, istemcinin beklediği biçime çeviren aracı sınıf.

Şimdi de desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <memory> // std::unique_ptr için

// 1. Adaptee (Uyarlanan Uyumsuz Arayüz): Mevcut olan ama arayüzü uyumsuz olan sınıf.
class Adaptee {
public:
    void specificRequest() const {
        std::cout << "Adaptee: Özel istek (specificRequest) yontemi calistirildi." << std::endl;
    }
};

// 2. Target (Hedef): İstemcinin (Client) beklediği arayüz.
class Target {
public:
    // ÖNEMLİ: Soyut sınıflarda sanal yıkıcı (virtual destructor) şarttır!
    virtual ~Target() = default;
    virtual void request() = 0;
};

// 3. Adapter (Uyarlayıcı): Target arayüzünü Adaptee'ye bağlar.
class Adapter : public Target {
private:
    // Akıllı işaretçi ile otomatik ömür yönetimi (RAII)
    std::unique_ptr<Adaptee> adaptee;
public:
    // make_unique ile güvenli oluşturma
    Adapter() : adaptee(std::make_unique<Adaptee>()) {}
    void request() override {
        std::cout << "Adapter: Standart istek alındı, Adaptee'ye yonlendiriliyor..." <<
            << std::endl;
        adaptee->specificRequest();
    }
};

// 4. Client (İstemci)
int main() {
    // Target nesnesini akıllı işaretçi ile yönetiyoruz.
    // Nesne kapsam dışına çıktığında delete işlemi otomatik yapılır.
    std::unique_ptr<Target> target = std::make_unique<Adapter>();
    target->request();
}

/*Program Çıktısı:
Adapter: Standart istek alındı, Adaptee'ye yonlendiriliyor...
Adaptee: Özel istek (specificRequest) yontemi calistirildi.
*/
```

Bu desen gerçek dünya örneği olarak aşağıdaki iz kaydı (**log**) örneği verilebilir;

```
//C++ 14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <memory>

//1. --- Adaptee (Uyarlanan Sınıf) ---
// Değiştiremediğimiz veya eski (Legacy) olan sınıf
class OldLogger {
public:
    void logMessage(const char* msg) {
```

```

        std::cout << "[Eski Kayit Sistemi]: " << msg << std::endl;
    }
};

//2. --- Target Interface (Hedef Arayüz) ---
// Yeni sistemin beklediği standart arayüz
class ILogger {
public:
    virtual ~ILogger() = default;
    virtual void log(const std::string& msg) = 0;
};

//3. --- Adapter (Adaptör Sınıfı) ---
// Eski sınıfı yeni arayüze sarmalayan dönüştürücü
class LoggerAdapter : public ILogger {
private:
    std::unique_ptr<OldLogger> oldLogger_; // Nesne ömrünü yönetmek için akıllı işaretçi
public:
    // Adaptör oluşturulurken eski sınıfın bir örneğini alır
    LoggerAdapter(std::unique_ptr<OldLogger> logger) : oldLogger_(std::move(logger)) {}
    // Yeni arayüz metodu çağrıldığında, arka planda eski metodu uygun parametreyle çalıştırır
    void log(const std::string& msg) override {
        // std::string'i eski sistemin beklediği const char*'a çeviriyoruz
        oldLogger_>logMessage(msg.c_str());
    }
};

//4. --- Client (İstemci) ---
// Sadece ILogger arayüzünü tanır
void appLog(ILogger& logger) {
    logger.log("Sistem basariyla calisti.");
}

int main() {
    // 1. Eski sistemden bir logger nesnesi oluşturalım
    auto oldSys = std::make_unique<OldLogger>();
    // 2. Bu nesneyi yeni sistemde kullanamayız çünkü metod isimleri ve tipleri farklı.
    // Bu yüzden Adaptör devreye giriyor:
    std::unique_ptr<ILogger> modernLogger =
        std::make_unique<LoggerAdapter>(std::move(oldSys));
    // 3. İstemci (appLog), arka planda eski bir sistemin çalıştığını bilmeden
    // standart log() metodunu çağırır.
    std::cout << "Modern uygulama log gönderiyor..." << std::endl;
    appLog(*modernLogger);
}

/* Programın Çıktısı:
Modern uygulama log gönderiyor...
[Eski Kayit Sistemi]: Sistem basariyla calisti.
*/

```

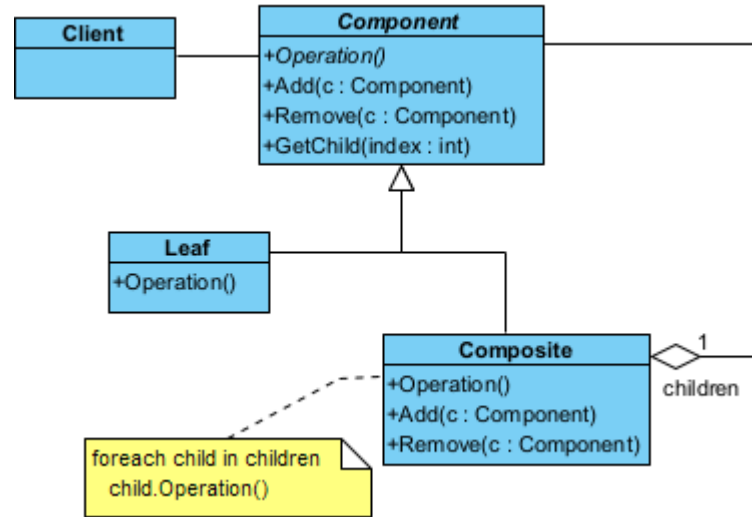
§17.3.3 Bileşik

Bileşik deseni (**composite pattern**), özyinelemeli (**recursive**) ya da hiyerarşik yapıya sahip nesne oluşturmak gerektiğinde yapısal olarak nasıl bir kodlama yöntemi izleneceğini söyler. Bu desen, her nesnenin bağımsız olarak veya aynı arayüz üzerinden iç içe geçmiş nesneler kümesi olarak ele alınabileceği nesne hiyerarşilerinin oluşturulmasını kolaylaştırır.

Bu desenin görevi, nesneleri "parça-bütün" hiyerarşisinde (ağaç yapısı) düzenler. Tekil nesne ile nesne gruplarına aynı şekilde davranılmasını sağlar. Bir klasör örneği üzerinden ilerlersek;

- ◊ Bileşen (**Component**), hem dosyaların hem klasörlerin ortak özelliklerini tanımlayan arayüz.
- ◊ Yaprak (**Leaf**), En küçük birim (Örnek olarak Dosya).
- ◊ Bileşik (**Composite**), içinde başka birimler barındırabilen yapı (Örnek olarak Dosya içerebilen Klasör).

Şimdi de desenimizi kodlayalım;



Şekil 17.8: Bileşik Deseni UML Diyagramı

```

//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <list>
#include <memory> // unique_ptr ve make_unique için
#include <string>
#include <algorithm>

//1.--- Abstract Component ---
class Component {
protected:
    Component* ebeveyn = nullptr; // Sadece gözlem amaçlı ham pointer (sahiplik içermez)
public:
    virtual ~Component() = default;
    void setEbeveyn(Component* pEbeveyn) {
        this->ebeveyn = pEbeveyn;
    }
    Component* getEbeveyn() const {
        return this->ebeveyn;
    }
    virtual bool IsComposite() const {
        return false;
    }
    // Sahipliği devralmak için unique_ptr kullanıyoruz
    virtual void addChild(std::unique_ptr<Component> component) {}
    virtual void removeChild(Component* component) {}
    virtual std::string displayOperation() const = 0;
};

//2.--- Leaf (Yaprak) ---
class Leaf : public Component {
public:
    std::string displayOperation() const override {
        return "Yaprak";
    }
};

//3.--- Composite (Dal) ---
class Composite : public Component {
private:
    // Çocukların gerçek sahibi bu listedir
    std::list<std::unique_ptr<Component>> cocuklar;
public:
    void addChild(std::unique_ptr<Component> pComponent) override {

```

```

    pComponent->setEbeveyn(this);
    this->cocuklar.push_back(move(pComponent)); // Sahiplik listeye geçer
}
void removeChild(Component* pComponent) override {
    // Listede bu adrese sahip olan unique_ptr'yi bul ve sil
   ocuklar.remove_if([pComponent](const std::unique_ptr<Component>& ptr) {
        return ptr.get() == pComponent;
    });
}
bool IsComposite() const override {
    return true;
}
std::string displayOperation() const override {
    std::string result;
    for (auto it = cocuklar.begin(); it != cocuklar.end(); ++it) {
        if (it != cocuklar.begin()) result += "+";
        result += (*it)->displayOperation();
    }
    return "Dal(" + result + ")";
}
};

// 4.--- Client (İstemci) ---
int main() {
    // 1. Sadece yaprak örneği
    auto yaprak = std::make_unique<Leaf>();
    std::cout << "Sadece Yapraktan Olusan Bilesik Nesne:" ;
    std::cout << yaprak->displayOperation() << std::endl;
    // 2. Karmaşık ağaç yapısı
    auto kok = std::make_unique<Composite>();
    auto dal1 = std::make_unique<Composite>();
    // dal1'i kok'e eklemeyen önce yaprakları dal1'e ekleyelim
    dal1->addChild(std::make_unique<Leaf>()); // yaprak1
    // Daha sonra silmek istediğimiz bir yaprak varsa adresini saklamalıyız
    auto yaprak2_raw = new Leaf(); // Geçici ham pointer (veya make_unique sonrası .get())
    dal1->addChild(std::unique_ptr<Leaf>(yaprak2_raw));
    kok->addChild(std::move(dal1)); // dal1'in sahipliği kok'e geçti
    std::cout << "Yapraktan ve Dallardan Olusan Bilesik Nesne:" ;
    std::cout << kok->displayOperation() << std::endl;
    // 3. Silme işlemi (yaprak2_raw üzerinden güvenli bir şekilde)
    // Not: kok içindeki dal1'e erişmek için hiyerarşiyi bilmek gerekir,
    // gerçek projelerde daha dinamik yöntemler kullanılır.
    // Burada basitlik adına doğrudan kok üzerinden gösterim yapıyoruz.
    // 4. Dal2 ekleme
    auto dal2 = std::make_unique<Composite>();
    dal2->addChild(std::make_unique<Leaf>()); // yaprak3
    kok->addChild(std::move(dal2));
    std::cout << "Bir dal daha eklenmiş Bilesik Nesne:" ;
    std::cout << kok->displayOperation() << std::endl;
    // Hiçbir delete işlemine gerek yok!
    // 'kok' silindiğinde tüm dallar ve yapraklar sırayla otomatik silinir.
}
/* Programın Çıktısı:
Sadece Yapraktan Olusan Bilesik Nesne:Yaprak
Yapraktan ve Dallardan Olusan Bilesik Nesne:Dal(Dal(Yaprak+Yaprak))
Bir dal daha eklenmiş Bilesik Nesne:Dal(Dal(Yaprak+Yaprak)+Dal(Yaprak))
*/

```

Bu desene gerçek dünya örneği olarak aşağıdaki dosya sistemini gösteren örnek verilebilir;

```

//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>

```

```

#include <vector>
#include <memory>
#include <utility>

//1. --- Component (Bileşen) ---
class FileSystemNode {
public:
    virtual ~FileSystemNode() = default;
    // Varsayılan parametreyi burada tanımlıyoruz
    virtual void showDetails(int indent = 0) const = 0;
    virtual long getSize() const = 0;
};

//2. --- Leaf (Yaprak) ---
class File : public FileSystemNode {
private:
    std::string name_;
    long size_;
public:
    File(std::string name, long size) : name_(std::move(name)), size_(size) {}
    // override kelimesiyle imzanın aynı olduğunu garantiliyoruz
    void showDetails(int indent) const override {
        std::string spacing(indent, ' ');
        std::cout << spacing << "- Dosya: " << name_ << " (" << size_ << " KB)" << std::endl;
    }
    long getSize() const override { return size_; }
};

//3. --- Composite (Bileşik) ---
class Directory : public FileSystemNode {
private:
    std::string name_;
    std::vector<std::shared_ptr<FileSystemNode>> children_;
public:
    Directory(std::string name) : name_(std::move(name)) {}
    void add(std::shared_ptr<FileSystemNode> node) {
        children_.push_back(node);
    }
    // Dikkat: Burada 'indent = 0' yazmaya gerek yok, base class'tan devralınır.
    void showDetails(int indent) const override {
        std::string spacing(indent, ' ');
        std::cout << spacing << "+ Klasör: " << name_ << " [Toplam: " << getSize() << " KB]" <<
        << std::endl;
        for (const auto& child : children_) {
            // Özyinelemeli çağrı
            child->showDetails(indent + 4);
        }
    }
    long getSize() const override {
        long total = 0;
        for (const auto& child : children_) {
            total += child->getSize();
        }
        return total;
    }
};

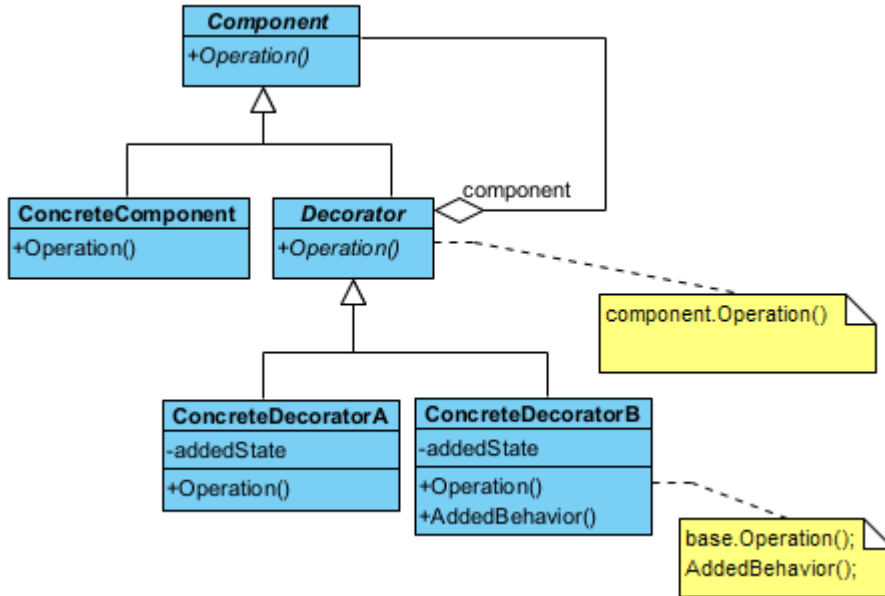
//4. --- Client (İstemci) ---
int main() {
    // Klasörleri oluşturalım
    auto root = std::make_shared<Directory>("C_Surucusu");
    auto docs = std::make_shared<Directory>("Belgelerim");

```

```
// Dosyaları ekleyelim
docs->add(std::make_shared<File>("Notlar.txt", 10));
root->add(docs);
root->add(std::make_shared<File>("Sistem.log", 500));
// Bazı derleyiciler paylaşılan pointer (shared_ptr) üzerinden varsayılan parametreye
// erişirken sorun yaşayabilir. Açıkça 0 göndererek veya arayüzü doğru kullanarak çözeriz.
root->showDetails(0);
}
/* Programın Çıktısı:
+ Klasor: C_Surucusu [Toplam: 510 KB]
+ Klasor: Belgelerim [Toplam: 10 KB]
- Dosya: Notlar.txt (10 KB)
- Dosya: Sistem.log (500 KB)
*/
```

§17.3.4 Süsleyici

Süsleyici deseninde (decorator pattern), bir nesneye dinamik olarak yeni durum ve davranışları eklemek mümkündür. Yani çalıştırma anında, bir nesnenin sahip olduğu yeteneklere, yeni yeteneklerin eklenmesini sağlar.



Şekil 17.9: Süsleyici Deseni UML Diyagramı

Bu tasarım deseni, bir nesneye dinamik olarak (çalışma zamanında) yeni sorumluluklar veya özellikler eklemek için kullanılan yapısal bir desendir. Kalıtımın (**inheritance**) aksine, özellikleri sınıf hiyerarşisini karmaşıktırmadan, nesneleri birbirinin içine "sarmalayarak" (**wrapping**) ekler. Bunu bir Rus matruşka bebeği veya bir kahve siparişine eklenen ekstralar (süt, şeker, karamel) gibi düşünebilirsiniz.

- ◊ Bileşen (**Component**), hem temel nesnenin hem de süsleyicilerin uyması gereken ortak soyut arayüzü tanımlar. Temel işlevin (Örnek olarak `ucretHesapla()`) imzasını tanımlar.
- ◊ Somut Bileşen (**ConcreteComponent**), üzerine ekleme yapılacak olan "temel" nesnedir. Fonksiyonun en yalın halini gerçekleştirir (Örnek olarak Sade Kahve).
- ◊ Soyut Süsleyici (**Decorator**), süsleyicilerin soyut temel yapısını kurar. İçinde bir **Component** referansı tutar. Bu referans sayesinde hem temel nesneyi hem de başka bir süsleyiciyi sarmalayabilir.
- ◊ Somut Süsleyiciler (**ConcreteDecorator**), gerçek özellikleri ekleyen sınıflardır. Temel nesnenin metodunu çağırır ve üzerine kendi eklemesini yapar (Örnek olarak Kahve ücretine +5 TL süt bedeli eklemek).

Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmeli
#include <iostream>
#include <string>
#include <memory> // std::unique_ptr için

// 1. Component (Bileşen): Ortak arayüz
```

```

class Component {
public:
    virtual ~Component() = default; // Kritik: Sanal yıkıcı
    virtual void operation() const = 0;
};

// 2. ConcreteComponent (Somut Bileşen): Temel nesne
class ConcreteComponent : public Component {
public:
    void operation() const override {
        std::cout << "Temel Urun Islemi" << std::endl;
    }
};

// 3. Decorator (Süsleyici): Sarmalayıcı temel sınıf
class Decorator : public Component {
protected:
    std::unique_ptr<Component> component; // Sahiplik dekoratördedir
public:
    Decorator(std::unique_ptr<Component> c) : component(std::move(c)) {}
    void operation() const override {
        if (component) component->operation();
    }
};

// 4. ConcreteDecorator1: Yeni Durum Ekleyen Süsleyici
class ConcreteDecorator1 : public Decorator {
private:
    int addedState;
public:
    ConcreteDecorator1(std::unique_ptr<Component> c, int state)
        : Decorator(std::move(c)), addedState(state * 100) {}
    void operation() const override {
        Decorator::operation(); // Önceki nesnenin davranışını çağır
        std::cout << "-> Decorator 1: Eklendi (State: " << addedState << ")" << std::endl;
    }
};

// 5. ConcreteDecorator2: Yeni Davranış Ekleyen Süsleyici
class ConcreteDecorator2 : public Decorator {
public:
    ConcreteDecorator2(std::unique_ptr<Component> c) : Decorator(std::move(c)) {}
    std::string addedBehavior() const {
        return "Bu metin Yeni Davranıştan Geliyor...";
    }
    void operation() const override {
        Decorator::operation(); // Önceki nesnenin davranışını çağır
        std::cout << "-> Decorator 2: Yeni Davranış Tetiklendi: " << addedBehavior() << std::endl;
    }
};

// 6. Client (İstemci)
int main() {
    // Nesneleri iç içe sararak (stacking) oluşturuyoruz
    std::cout << "--- Dekorator Zinciri Olusturuluyor ---" << std::endl;
    // 1. Temel ürün oluştur
    auto myProduct = std::make_unique<ConcreteComponent>();
    // 2. Ürünü Decorator 1 ile süsle
    auto decorated1 = std::make_unique<ConcreteDecorator1>(std::move(myProduct), 2);
    // 3. Ortaya çıkan nesneyi Decorator 2 ile tekrar süsle
    auto decorated2 = std::make_unique<ConcreteDecorator2>(std::move(decorated1));
}

```

```

    // Tek bir çağrı tüm zinciri tetikler
    decorated2->operation();
    // Her şey otomatik silinir (unique_ptr sayesinde)
}
/*Program Çıktısı:
--- Dekorator Zinciri Olusturuluyor ---
Temel Urun Islemi
-> Decorator 1: Eklendi (State: 200)
-> Decorator 2: Yeni Davranis Tetiklendi: Bu metin Yeni Davranistan Geliyor...
*/

```

Aşağıda gerçek dünya örneği olarak sipariş edilen kahveyi süsleyen bir örnek verilmiştir;

```

//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <memory> // unique_ptr ve move için gerekli
#include <utility> // std::move için

//1. --- Base Component (Temel Bileşen) ---
class Coffee {
public:
    virtual ~Coffee() = default;
    virtual double cost() const = 0;
    virtual std::string description() const = 0;
};

//2. --- Concrete Component (Somut Bileşen) ---
class SimpleCoffee : public Coffee {
public:
    double cost() const override { return 5.0; }
    std::string description() const override { return "Sade Kahve"; }
};

//3. --- Base Decorator (Temel Süsleyici) ---
class CoffeeDecorator : public Coffee {
protected:
    std::unique_ptr<Coffee> coffee_;
public:
    CoffeeDecorator(std::unique_ptr<Coffee> coffee) : coffee_(std::move(coffee)) {}
};

//4. --- Concrete Decorators (Somut Süsleyiciler) ---
class MilkDecorator : public CoffeeDecorator {
public:
    using CoffeeDecorator::CoffeeDecorator;
    double cost() const override {
        return coffee_->cost() + 1.5; // Mevcut fiyata süt ekle
    }
    std::string description() const override {
        return coffee_->description() + ", süt";
    }
};

class SugarDecorator : public CoffeeDecorator {
public:
    using CoffeeDecorator::CoffeeDecorator;
    double cost() const override {
        return coffee_->cost() + 0.5; // Mevcut fiyata şeker ekle
    }
    std::string description() const override {
        return coffee_->description() + ", seker";
    }
}

```

```

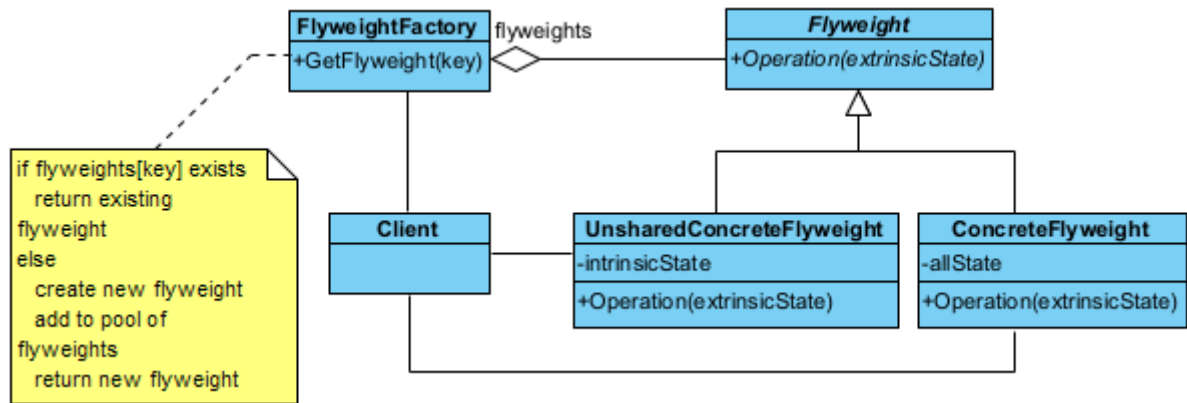
};

//5. --- Client (İstemci) ---
int main() {
    // 1. Sade bir kahve oluşturuyoruz
    std::unique_ptr<Coffee> kahvem = std::make_unique<SimpleCoffee>();
    // 2. Kahveye süt ekliyoruz (Sade kahveyi MilkDecorator ile sarmalıyoruz)
    kahvem = std::make_unique<MilkDecorator>(std::move(kahvem));
    // 3. Kahveye şeker ekliyoruz (Sütlü kahveyi SugarDecorator ile sarmalıyoruz)
    kahvem = std::make_unique<SugarDecorator>(std::move(kahvem));
    // Sonuçları yazdıralım
    std::cout << "Siparis: " << kahvem->description() << std::endl;
    std::cout << "Toplam Fiyat: " << kahvem->cost() << " TL" << std::endl;
}
/*
Siparis: Sade Kahve, süt, seker
Toplam Fiyat: 7 TL
*/

```

§17.3.5 Sinek Sıklet

Sinek Sıklet deseninde (flyweight pattern), çok sayıda benzer nesnenin bulunduğu durumlarda bellek kullanımını minimize etmek için kullanılan yapısal (structural) bir desendir.



Şekil 17.10: Sinek Sıklet Deseni UML Diyagramı

Bu desenin temel stratejisi, nesnelerin ortak verilerini (**intrinsic state**) paylaşarak bellekte tek bir kopyasını tutmak, her nesneye özgü değişen verileri (**extrinsic state**) ise dışarıdan sağlamaktır.

- ◊ Soyut Sinek Sıklet (**Flyweight**), paylaşılan nesnelerin uyması gereken arayüzü tanımlar. Nesneye özgü verilerin (**extrinsic state**) parametre olarak geçildiği yöntemin imzasını barındırır.
- ◊ Somut Sinek Sıklet (**ConcreteFlyweight**), paylaşılan veriyi (**intrinsic state**) kendi içinde saklayan nesnedir. Bu nesneler "değişmez" (**immutable**) olmalıdır. Örneğin; bir oyun için "Ağaç" modelinin 3D verisi burada bir kez tutulur.
- ◊ Sinek Sıklet Fabrikası (**FlyweightFactory**), **Flyweight** nesnelerini yönetir ve paylaşılmasını sağlar. İstendiğinde, eğer nesne daha önce oluşturulmuşsa mevcut olanı verir; yoksa yenisini oluşturup havuza ekler.
- ◊ Bağlam (**Context**), her nesneye özgü, sürekli değişen verileri (konum, renk gibi) tutar. Somut **ConcreteFlyweight** nesnesini bu verilerle birlikte kullanarak asıl işlemi gerçekleştirir.

Şimdi desenimizi kodlayalım;

```

//C++17 ve üzeri ile derlenecek
#include <iostream>
#include <string>
#include <vector>
#include <unordered_map>
#include <memory>

// 1. Flyweight Arayüzü
class Flyweight {
public:

```



```

    virtual ~Flyweight() = default;
    virtual void operation(const std::string& extrinsicState) const = 0;
protected:
    std::string intrinsicState; // Ortak olan, değişmez durum
    Flyweight(std::string state) : intrinsicState(std::move(state)) {}
};

// 2. Somut Paylaşılan Flyweight (Concrete Flyweight)
class ConcreteFlyweight : public Flyweight {
public:
    explicit ConcreteFlyweight(std::string state) : Flyweight(std::move(state)) {}

    void operation(const std::string& extrinsicState) const override {
        std::cout << "[Paylaşılan] Ic Durum: " << intrinsicState
        << " | Dis Durum: " << extrinsicState << std::endl;
    }
};

// 3. Flyweight Factory (Fabrika)
class FlyweightFactory {
private:
    // Hızlı erişim için map kullanıyoruz. Nesne ömrünü shared_ptr yönetir.
    std::unordered_map<std::string, std::shared_ptr<Flyweight>> flyweights;
public:
    std::shared_ptr<Flyweight> getFlyweight(const std::string& key) {
        // Eğer bu anahtara sahip nesne zaten varsa onu döndür
        if (flyweights.find(key) != flyweights.end()) {
            std::cout << "-> Fabrika: " << key << " zaten var, mevcut olan döndürülüyor.\n";
            return flyweights[key];
        }

        // Yoksa yeni bir tane oluştur ve depola
        std::cout << "-> Fabrika: " << key << " bulunamadi, yeni olusturuluyor.\n";
        auto flyweight = std::make_shared<ConcreteFlyweight>(key);
        flyweights[key] = flyweight;
        return flyweight;
    }

    void listFlyweights() const {
        std::cout << "\nFabrikadaki toplam nesne sayisi: " << flyweights.size() << "\n";
        for (const auto& [key, value] : flyweights) {
            std::cout << "- " << key << "\n";
        }
    }
};

// 4. Client (İstemci)
int main() { // Client (İstemci)
    FlyweightFactory factory;
    // Aynı nesneleri tekrar tekrar isteyelim
    auto v1 = factory.getFlyweight("Binek Arac");
    auto v2 = factory.getFlyweight("Kamyon");
    auto v3 = factory.getFlyweight("Binek Arac"); // Fabrikadan mevcut olan gelecek
    // Dışsal durumları (konum, plaka gibi) işlem sırasında gönderiyoruz
    v1->operation("Konum: 41.00, 28.97");
    v2->operation("Konum: 39.93, 32.85");
    v3->operation("Konum: 40.71, -74.00");
    factory.listFlyweights();
    // shared_ptr sayesinde manuel delete gerekmez.
}

/*Program Çıktısı:
-> Fabrika: Binek Arac bulunamadi, yeni olusturuluyor.
-> Fabrika: Kamyon bulunamadi, yeni olusturuluyor.
-> Fabrika: Binek Arac zaten var, mevcut olan döndürülüyor.
[Paylaşılan] Ic Durum: Binek Arac | Dis Durum: Konum: 41.00, 28.97
[Paylaşılan] Ic Durum: Kamyon | Dis Durum: Konum: 39.93, 32.85
*/

```

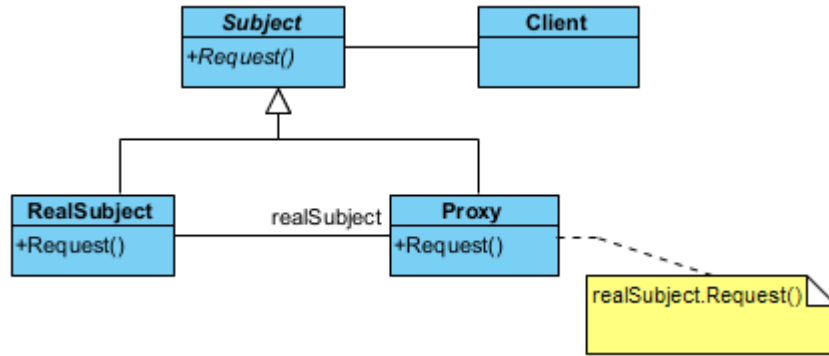
[Paylaşılan] Ic Durum: Binek Arac | Dis Durum: Konum: 40.71, -74.00

Fabrikadaki toplam nesne sayısı: 2

- Kamyon
- Binek Arac
*/

§17.3.6 Vekil

Vekil deseni (proxy pattern), kullanılacak ve erişilecek bir nesnenin yerine geçen bir başka nesneye ihtiyaç duyulduğunda kullanılır. Bu desen, başka bir nesneye erişimi kontrol etmek için kullanılan yapısal bir desendir. Bir nesnenin önüne "vekil" bir katman koyarak, asıl nesneye erişmeden önce güvenlik kontrolü yapılabilir, nesnenin yüklenmesini geciktirebilir (**lazy loading**) veya işlem sonuçlarını önbelleğe alabilirsiniz.



Şekil 17.11: Vekil Deseni UML Diyagramı

Bu deseni bir şirketteki "Yönetici Asistanı" gibi düşünebilirsin. Yöneticiyle doğrudan görüşemezsin; önce asistanla (**Proxy**) görüşürsün, asistan uygun görürse seni yöneticiye (**RealSubject**) yönlendirir.

- ◊ Bağlam (**Subject**), hem asıl nesnenin hem de vekilin (**Proxy**) uyması gereken ortak soyut arayüzü tanımlar. İstemcinin (**Client**) her iki nesneye de aynı şekilde davranabilmesini sağlar.
- ◊ Gerçek Bağlam (**RealSubject**), asıl işi yapan, asıl mantığı barındıran nesnedir. Genellikle oluşturulması maliyetli veya erişimi kısıtlı olan nesnedir.
- ◊ Vekil (**Proxy**), gerçek nesneye olan referansı kendi içinde saklar. İstekleri gerçek nesneye iletmeden önce araya girer. Erişim kontrolü yapabilir, gerçek nesne henüz oluşturulmamışsa onu oluşturabilir veya gelen isteği reddedebilir.
- ◊ İstemci (**Client**), nesnenin doğrudan kendisine değil, vekil (**Proxy**) nesnesine ulaşır ve bu nesneye isteklerini iletir.

Şimdi desenimizi kodlayalım;

```

//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <memory> // std::unique_ptr için

// 1. Subject (Özne Arayüzü): Gerçek nesne ve vekilin ortak arayüzü
class Subject {
public:
    virtual ~Subject() = default; // Kritik: Modern C++ sanal yıkıcı
    virtual void request() const = 0;
};

// 2. RealSubject (Gerçek Özne): Asıl işi yapan nesne
class RealSubject : public Subject {
private:
    int state;
public:
    explicit RealSubject(int pState) : state(pState) {}
    void request() const override {
        std::cout << "RealSubject: İstek isleniyor (Durum: " << state << ")" << std::endl;
    }
};
  
```

```
// 3. Proxy (Vekil): Gerçek nesneye erişimi yönetir
class Proxy : public Subject {
private:
    // Mülkiyet vekilin içindedir (RAII)
    std::unique_ptr<RealSubject> realSubject;
    bool checkAccess() const {
        std::cout << "Proxy: Erisim yetkisi kontrol ediliyor..." << std::endl;
        return true;
    }
    void logAccess() const {
        std::cout << "Proxy: Islem gunlugu (log) kaydedildi." << std::endl;
    }
public:
    // Senaryo 1: Vekil kendi nesnesini olusturur (Lazy Initialization örneği)
    Proxy() : realSubject(std::make_unique<RealSubject>(20)) {}
    // Senaryo 2: Mevcut bir nesnenin kopyası üzerinden vekillik yapar
    explicit Proxy(const RealSubject& source) :
        realSubject(std::make_unique<RealSubject>(source)) {}
    void request() const override {
        if (checkAccess()) {
            realSubject->request();
            logAccess();
        }
    }
};

// 4. Client (İstemci)
int main() { // Client (İstemci)
    // 1. Doğrudan erişim
    std::cout << "--- Dogrudan RealSubject Kullanimi ---" << std::endl;
    auto gercekOzne = std::make_unique<RealSubject>(10);
    gercekOzne->request();
    // 2. Kendi nesnesini yöneten vekil (Default Proxy)
    std::cout << "--- Proxy (Yeni Nesne Ile) ---" << std::endl;
    auto vekil1 = std::make_unique<Proxy>();
    vekil1->request();
    // 3. Mevcut nesneyi kopyalayan vekil
    std::cout << "--- Proxy (Mevcut Nesne Kopyasi Ile) ---" << std::endl;
    auto vekil2 = std::make_unique<Proxy>(*gercekOzne);
    vekil2->request();
    // Not: Manuel 'delete' yok! unique_ptr sayesinde her şey otomatik temizlenir.
}

/*Program Çıktısı:
--- Dogrudan RealSubject Kullanimi ---
RealSubject: Istek isleniyor (Durum: 10)
--- Proxy (Yeni Nesne Ile) ---
Proxy: Erisim yetkisi kontrol ediliyor...
RealSubject: Istek isleniyor (Durum: 20)
Proxy: Islem gunlugu (log) kaydedildi.
--- Proxy (Mevcut Nesne Kopyasi Ile) ---
Proxy: Erisim yetkisi kontrol ediliyor...
RealSubject: Istek isleniyor (Durum: 10)
Proxy: Islem gunlugu (log) kaydedildi.
*/
```

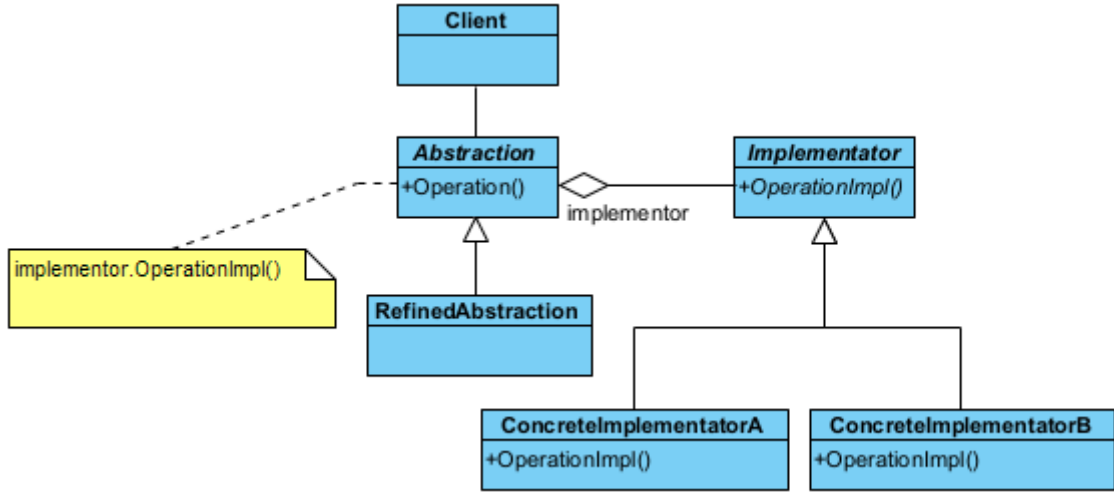
Bu desenle ilgili şu kavramlardan bahsetmek gerekir;

- ◊ Bir nesne, bulunduğu yerden uzaktaki bir nesneyi kullanacaksa uzak vekil (**remote proxy**) kullanılır. Gerçek nesne çok ağır (örneğin büyük bir resim), nesneyi sadece gerçekten ihtiyaç duyulduğunda (**lazy loading**) oluşturur.
- ◊ Bir nesneye başka nesneler tarafından erişim kısıtlanacaksa koruma vekili (**protection proxy**) olarak adlandırılır. Yetkilendirme (**authentication**) kontrolü yapar.
- ◊ Bir nesneye erişim sırasında başka işlemler yapılacaksa vekil, akıllı referans (**smart reference**) olarak

adlandırılır. Böyle durumda vekil, nesneye bir erişim yapıldığında, diğer nesnelerin erişmemesi için kullanılacak nesneye kilit koyabilir. Akıllı referans, akıllı gösterici (**smart pointer**) olarak da adlandırılır. Nesne başka bir sunucudaysa ağ trafiğini yönetir.

§17.3.7 Köprü

Köprü deseni (bridge pattern), işi yapan alt yüklenici veya taşıeron (**implementator**) nesne ile işi yaptıran nesne (**client**) arasındaki bağımlılığı soyutlama ile ortadan kaldırır. Yani birbiriyle oldukça iç içe olan kodlama ile soyutlamayı ortak arayüz ile birbirinden uzaklaştırır. Sistemin genişlemesine izin verir. Bu desen, yazılımlarda oldukça çok kullanılan bir desendir. Çünkü taşıeronlar değişse de yeni taşıeron eklense de istemci tarafında bir şey değişmez.



Şekil 17.12: Köprü Deseni UML Diyagramı

Bu tasarım deseni, bir nesnenin soyutlamasını (**abstraction**), onun uygulamasından (**implementation**) ayırarak her ikisinin de birbirinden bağımsız olarak geliştirilmesini sağlayan yapısal bir desendir. Bu desen, özellikle **sınıf patlaması (class explosion)** dediğimiz, her yeni özellik için onlarca alt sınıf türetmek zorunda kaldığımız durumlarda imdadımıza yetişir. Kalıtım yerine bileşimi (**composition**) kullanır.

- ◇ Soyutlama (**Abstraction**), istemcinin (**Client**) kullandığı üst düzey kontrol arayüzüdür. İçinde bir **Implementor** referansı tutar. İstemciden gelen komutları bu referansa iletir.
- ◇ Geliştirilmiş Soyutlama (**RefinedAbstraction**), soyutlamayı genişleten alt sınıflardır. Temel arayüzü kullanarak daha spesifik kontrol mantıkları ekler (Örn: Standart Kumanda -> Gelişmiş Akıllı Kumanda).
- ◇ Taşıeron Arayüzü (**Implementor**), tüm somut uygulama sınıfları için ortak arayüzü tanımlar. Genellikle düşük seviyeli (ilkel) operasyonları barındırır (Örn: **cihazIac()**, **sesAyari()**).
- ◇ Somut Taşıeronlar (**ConcreteImplementor**), **Implementor** arayüzünü gerçekleyen asıl cihazlardır. Platforma veya cihaza özgü gerçek kodları barındırır (Örn: Televizyon, Radyo).

Şimdi desenimizi kodlayalım;

```

//C++14 ve üzeri ile derlenmeli
#include <iostream>
#include <memory> // Akıllı işaretçiler için
#include <utility>

//1.--- Implementor (Uygulayıcı/Taşıeron Arayüzü) ---
class Implementor {
public:
    virtual ~Implementor() = default; // Güvenli temizlik için sanal yıkıcı
    virtual void operationImpl() = 0;
};

//2.--- Concrete Implementors ---
class ConcreteImplementorA : public Implementor {
public:
    void operationImpl() override {
        std::cout << "A Taseronu tarafından yapılan is..." << std::endl;
    }
}

```

```

};
class ConcreteImplementorB : public Implementor {
public:
    void operationImpl() override {
        std::cout << "B Taseronu tarafından yapılan is..." << std::endl;
    }
};

//3.--- Abstraction (Soyut Yüklenici) ---
class Abstraction {
protected:
    // Taşeronun mülkiyeti (sahipliği) yükleniciye ait olsun veya olmasın,
    // burada shared_ptr veya unique_ptr seçimi stratejiktir.
    // Taşeronların çalışma anında değişebilirliği için unique_ptr ile sahipliği yönetiyoruz.
    std::unique_ptr<Implementor> implementor;
public:
    Abstraction(std::unique_ptr<Implementor> pImpl) : implementor(std::move(pImpl)) {}
    virtual ~Abstraction() = default;
    // Çalışma anında taşeronu güvenli bir şekilde değiştiriyoruz
    void setImplementor(std::unique_ptr<Implementor> pImpl) {
        implementor = move(pImpl);
    }
    void operation() {
        if (implementor) {
            std::cout << "Yuklenici Taserona is yaptiriyor:" << std::endl;
            implementor->operationImpl();
        }
    }
};

//4.--- Refined Abstraction (Geliştirilmiş Soyutlama) ---
class RefinedAbstraction : public Abstraction {
public:
    using Abstraction::Abstraction; // Üst sınıfın constructor'ını kullan
};

// 5.--- Client (İstemci) ---
int main() {
    // 1. Yükleniciyi ilk taşeronla (A) başlatıyoruz
    auto yuklenici =
        ↳ std::make_unique<RefinedAbstraction>(std::make_unique<ConcreteImplementorA>());
    yuklenici->operation();
    std::cout << "--- Taseron Degistiriliyor ---" << std::endl;
    // 2. Çalıştırma anında yeni bir taşeronla (B) geçiyoruz
    // Eski taşeron (A), unique_ptr'nin doğası gereği otomatik olarak hafızadan silinir.
    yuklenici->setImplementor(std::make_unique<ConcreteImplementorB>());
    yuklenici->operation();
    // Hiçbir delete işlemine gerek yok! 'yuklenici' kapsam dışına çıktığında içindeki taşeronla
    ↳ birlikte silinecektir.
}

/*Program Çıktısı:
Yuklenici Taserona is yaptiriyor:
A Taseronu tarafından yapılan is...
--- Taseron Degistiriliyor ---
Yuklenici Taserona is yaptiriyor:
B Taseronu tarafından yapılan is...
*/

```

Köprü deseni aşağıdaki durumlarda kullanılır;

- ◊ Taşeron nesneye kalıcı bir bağımlılık istenmediği durumlar. Çalışma anında (run-time) taşeronlar arasında geçiş yapmayı sağlar.
- ◊ Birden çok taşeron kullanarak sistemin genişlemesine ihtiyaç olduğu durumlar.

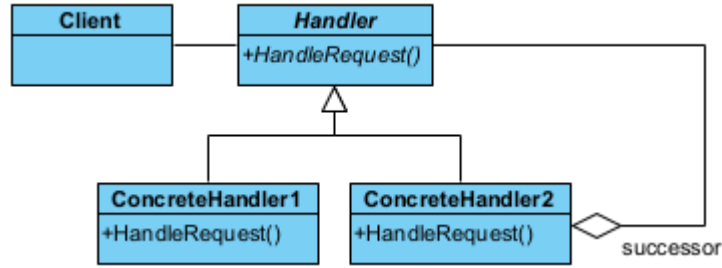
- ◇ Taşeronlar üzerindeki değişiklikler, istemciyi etkilemez. İstemci tarafında derleme olmadan yazılımın değiştirilmesi sağlanır.
- ◇ Projede aynı anda farklı birden çok taşeron tasarımı yapılabilirdiğinden, proje parçalara ayrılarak kendi içinde hızla sınıf tasarımı yapılabilir. Bu durum *Rumbaugh* tarafından iç içe genelleştirme (**nested generalization**) olarak adlandırılmıştır.

17.4 Davranışla İlgili Desenler

Davranışla ilgili desenler (**behavioral patterns**) nesnelerin çeşitli olaylar karşısında gösterecekleri davranışlara çözüm getirmektedirler.

§17.4.1 Sorumluluk Zinciri

Bazı durumlarda bir nesne, kendinin yapmayacağı bir işlemi halefine (**successor**) devredebilir. Ya da nesnenin kendi sorumluluğunu aşan durumlarda ilgili diğer nesneye işlem yaptırılır. İşte bu durumda **sorumluluk zinciri deseni** (**chain of responsibility pattern**) kullanılır.



Şekil 17.13: Sorumluluk Zinciri Deseni UML Diyagramı

Bu desenin temel amacı, isteği gönderen ile isteği işleyen arasındaki bağı gevşetmektir (**decoupling**). İstek, zincir boyunca ilerler ve her bir işleyici ya isteği çözer ya da bir sonraki işleyiciye paslar.

- ◇ İşleyici (**Handler**), İsteklerin nasıl işleneceğine dair ortak bir arayüz tanımlar. İşleyici, bir sonraki işleyiciyi tutan bir referans barındırır ve zincirin halkalarını birbirine bağlar.
- ◇ Somut İşleyiciler (**ConcreteHandler**), kendisine gelen isteği kontrol eder. Eğer istek kendi sorumluluk alanındaysa işi bitirir. Eğer değilse (veya işin bir kısmını yapıp devam etmesi gerekiyorsa), isteği zincirdeki bir sonraki halkaya iletir.
- ◇ İstemci (**Client**), zinciri oluşturur (halkaları birbirine bağlar) ve ilk isteği zincirin başına gönderir. İsteğin kim tarafından, nasıl çözüleceğiyle ilgilenmez; sadece "çözülmesini" bekler.

Şimdi desenimizi kodlayalım;

```

//C++14 ve üzeri ile derlenecek
#include <iostream>
#include <memory>

//1. Soyut İşleyici (Handler)
class Handler {
protected:
    std::unique_ptr<Handler> successor; // Zincirin bir sonraki halkası
public:
    virtual ~Handler() = default;
    // Zincire yeni bir halka eklemek için yardımcı metod
    void setSuccessor(std::unique_ptr<Handler> next) {
        successor = std::move(next);
    }
    virtual void handleRequest(int request) = 0;
};

//2. Somut İşleyici 1: Küçük Talepler (Örn: Memur)
class ConcreteHandler1 : public Handler {
public:
    void handleRequest(int request) override {
        if (request < 100) { //Birinci Halka: Kendisi
            std::cout << "Memur: " << request << " birimlik talebi onayladi." << std::endl;
        } else if (successor) { //İkinci Halka: Halefi

```

```

        std::cout << "Memur: Talep limitimi asiyor, Sefe devrediliyor..." << std::endl;
        successor->handleRequest(request);
    } else {
        std::cout << "Hata: Talebi karsilayacak kimse bulunamadi!" << std::endl;
    }
}

};

//3. Somut İşleyici 2: Büyük Talepler (Örn: Sef)
class ConcreteHandler2 : public Handler {
public:
    void handleRequest(int request) override {
        if (request >= 100 && request < 500) {
            std::cout << "Sef: " << request << " birimlik talebi onayladi." << std::endl;
        } else if (successor) {
            std::cout << "Sef: Talep limitimi asiyor, Mudure devrediliyor..." << std::endl;
            successor->handleRequest(request);
        } else {
            std::cout << "Sef: Bu kadar buyuk bir yetkim yok ve baska amir tanimli degil!" <<
                std::endl;
        }
    }
};

//4. Client (İstemci)
int main() {
    // Zinciri oluşturunuz
    auto memur = std::make_unique<ConcreteHandler1>();
    auto sef = std::make_unique<ConcreteHandler2>();
    // Memurun halefi Sef olsun
    memur->setSuccessor(std::move(sef));
    std::cout << "--- Talep 1 (50) ---" << std::endl;
    memur->handleRequest(50);
    std::cout << "--- Talep 2 (250) ---" << std::endl;
    memur->handleRequest(250);
    std::cout << "--- Talep 3 (1000) ---" << std::endl;
    memur->handleRequest(1000);
    // unique_ptr sayesinde tüm zincir otomatik olarak silinir.
}

/*Program Çıktısı:
--- Talep 1 (50) ---
Memur: 50 birimlik talebi onayladi.
--- Talep 2 (250) ---
Memur: Talep limitimi asiyor, Sefe devrediliyor...
Sef: 250 birimlik talebi onayladi.
--- Talep 3 (1000) ---
Memur: Talep limitimi asiyor, Sefe devrediliyor...
Sef: Bu kadar buyuk bir yetkim yok ve baska amir tanimli degil!
*/

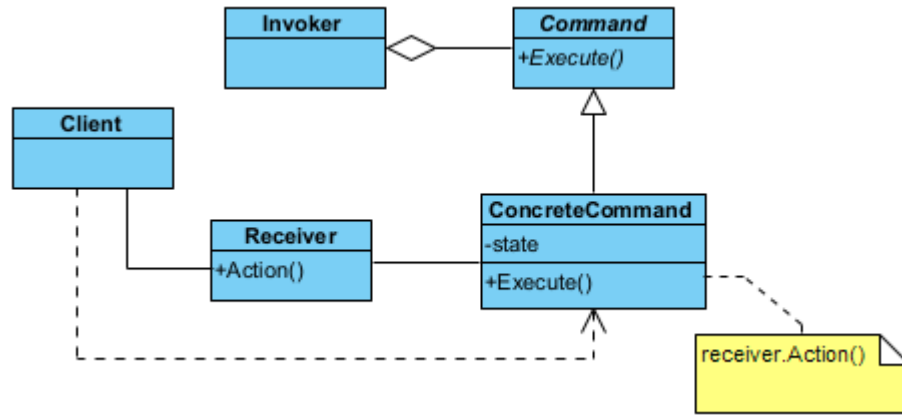
```

§17.4.2 Komut

Bazı durumlarda bir nesne diğer bir nesneye ileti (**message**) verip bir işlem başlattığında, bu işlem bitmeden başka işlemler yapmak isteyebilir. İşte bu tür durumlarda **komut deseni** (**command pattern**) kullanılır.

Bu tasarım deseni, bir isteği veya eylemi kendi başına bir nesneye dönüştüren davranışsal bir desendir. Bu dönüşüm sayesinde eylemleri parametre olarak geçebilir, kuyruğa alabilir (**queue**), iz kaydını tutabilir (**log**) veya geri alabilirsiniz (**undo**). Kısacası; "ne yapılacağı" bilgisini, "kimin yapacağı" bilgisinden tamamen ayırır.

- ◊ Komut (**Command**), tüm komutların uyması gereken soyut arayüzü tanımlar. Genellikle tek bir yöntem barındırır (Örnek olarak **execute()**). Bazı durumlarda geri alma işlemi için **undo()** yöntemini de içerir.
- ◊ Somut Komut (**ConcreteCommand**), soyut komutu gerçek bir eylemle birleştirir. Alıcı (**Receiver**) nesnesini tanıır ve **execute()** çağrıldığında alıcının ilgili metodunu tetikler.
- ◊ Alıcı(**Receiver**), işin asıl mutfağıdır. Gerçek mantığı ve operasyonları barındırır. Komut nesnesi ona



Şekil 17.14: Komut Deseni UML Diyagramı

"çalış" dediğinde, o işi nasıl yapacağını bilir (Örnek olarak Işığın açmak, dosyayı kaydetmek).

- ◊ Çağırıcı(Invoker), komutun ne zaman çalışacağına karar verir. Komutu saklar ve kullanıcı bir düğmeye bastığında veya bir olay gerçekleştiğinde komutun `execute()` metodunu çağırır. Alıcıyı doğrudan tanımaz.

Şimdi desenimizi kodlayalım;

//C++14 ve üzeri ile derlenecek

```
#include <iostream>
#include <string>
#include <vector>
#include <memory>
```

//1. Receiver (Alıcı): Asıl işi yapan sınıf

```
class Receiver {
public:
    void action(const std::string& pParam) const {
        std::cout << "Receiver: [" << pParam << "] komutu için temel işlemler yapılıyor.\n";
    }
    void actionA(const std::string& pParam) const {
        std::cout << "Receiver: [" << pParam << "] için A işlemi.\n";
    }
    void actionB(const std::string& pParam) const {
        std::cout << "Receiver: [" << pParam << "] için B işlemi.\n";
    }
};
```

//2. Command (Komut Arayüzü)

```
class Command {
public:
    virtual ~Command() = default;
    virtual void execute() const = 0;
};
```

//3. Concrete Commands (Somut Komutlar)

```
class SimpleCommand : public Command {
private:
    std::string state;
    Receiver& receiver; // Komut, mevcut bir alıcıya referansla bağlanır
public:
    SimpleCommand(std::string pState, Receiver& r) : state(std::move(pState)), receiver(r) {}
    void execute() const override {
        receiver.action(state);
    }
};

class ComplexCommand : public Command {
private:
```

```

    std::string state;
    Receiver& receiver;
public:
    ComplexCommand(std::string pState, Receiver& r) : state(std::move(pState)), receiver(r) {}
    void execute() const override {
        receiver.actionA(state);
        receiver.actionB(state);
    }
};

//4. Invoker (Çağırıcı): Komutları saklar ve tetikler
class Invoker {
private:
    std::vector<std::unique_ptr<Command>> history; // Komut geçmişini saklar
public:
    void storeAndExecute(std::unique_ptr<Command> cmd) {
        cmd->execute();
        history.push_back(std::move(cmd)); // Komutu sakla (Undo işlemleri için gerekebilir)
    }
};

//5. Client (İstemci)
int main() {
    // Tek bir alıcı (Receiver) oluşturulur
    Receiver receiver;
    Invoker invoker;
    std::cout << "--- Komutlar Tetikleniyor ---\n";
    // Komutlar oluşturulur ve Invoker aracılığıyla çalıştırılır
    invoker.storeAndExecute(std::make_unique<SimpleCommand>("Yazdir", receiver));
    invoker.storeAndExecute(std::make_unique<ComplexCommand>("Logla ve Gönder", receiver));
    // Her şey otomatik temizlenir
}

/*Program Çıktısı:
--- Komutlar Tetikleniyor ---
Receiver: [Yazdir] komutu için temel işlemler yapılıyor.
Receiver: [Logla ve Gönder] için A işlemi.
Receiver: [Logla ve Gönder] için B işlemi.
*/

```

Aşağıda komut desenine uygun bir ışıkım kapatılıp açılması örneği verilmiştir;

```

//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <vector>
#include <memory>
#include <string>

// --- Receiver (Alıcı) ---
class Light {
public:
    void on() { std::cout << "Isik ACILDI." << std::endl; }
    void off() { std::cout << "Isik KAPATILDI." << std::endl; }
};

// --- Command (Soyut Komut) ---
class Command {
public:
    virtual ~Command() = default;
    virtual void execute() = 0;
    virtual void undo() = 0;
};

// --- Concrete Commands (Somut Komutlar) ---

```

```

class LightOnCommand : public Command {
private:
    Light& light_; // Referans kullanımı: Işık nesnesi var olmak zorundadır.
public:
    explicit LightOnCommand(Light& light) : light_(light) {}
    void execute() override { light_.on(); }
    void undo() override { light_.off(); }
};

class LightOffCommand : public Command {
private:
    Light& light_;
public:
    explicit LightOffCommand(Light& light) : light_(light) {}
    void execute() override { light_.off(); }
    void undo() override { light_.on(); }
};

// --- Invoker (Çağırıcı) ---
class RemoteControl {
private:
    // Geçmişti tutan yığın (LIFO - Last In First Out)
    std::vector<std::unique_ptr<Command>> history_;
public:
    void executeCommand(std::unique_ptr<Command> cmd) {
        cmd->execute();
        history_.push_back(std::move(cmd));
    }
    void undoLast() {
        if (!history_.empty()) {
            // back() ile son komutu alıyoruz
            std::cout << "[UNDO] ";
            history_.back()->undo();
            // Komut işini bitirdi, artık hafızadan silebiliriz
            history_.pop_back();
        } else {
            std::cout << "[UYARI] Geri alınacak işlem yok!" << std::endl;
        }
    }
};

// --- Client (İstemci) ---
int main() {
    Light oturmaOdasiIsigi;
    RemoteControl kumanda;
    std::cout << "--- İşlemler Baslıyor ---" << std::endl;
    // Işığın açalım
    kumanda.executeCommand(std::make_unique<LightOnCommand>(oturmaOdasiIsigi));
    // Işığın kapatalım
    kumanda.executeCommand(std::make_unique<LightOffCommand>(oturmaOdasiIsigi));
    std::cout << std::endl << "--- Geri Alma (Undo) İşlemleri ---" << std::endl;
    kumanda.undoLast(); // Kapatmayı geri alır -> Işığın Açar
    kumanda.undoLast(); // Açmayı geri alır -> Işığın Kapatır
    kumanda.undoLast(); // Liste boş uyarısı verir
}

/*Program Çıktısı:
--- İşlemler Baslıyor ---
Isik ACILDI.
Isik KAPATILDI.

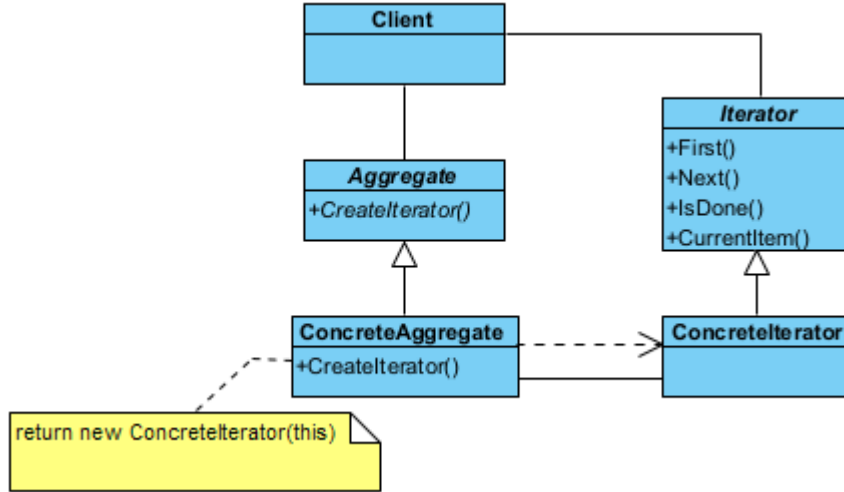
--- Geri Alma (Undo) İşlemleri ---
[UNDO] Isik ACILDI.
[UNDO] Isik KAPATILDI.
*/

```

```
[UYARI] Geri alınacak işlem yok!
*/
```

§17.4.3 Yineleyici

Yineleyici deseninde (**iterator pattern**), birden fazla elemandan oluşan küme (**aggregate**) içindeki her bir elemana sırayla erişim (iteration) sağlar. Erişim işleminde, istemcinin kümenin nasıl yapılandırıldığı konusunda bilgi sahibi olması gerekmez. Bu desende, istemci tarafından veri kümesi soyut (**abstract**) olarak kullanılmış olur. Yani istemciye, veri yapısından bağımsız bir şekilde verilere erişim sağlayan bir arayüz (**interface**) sunulur. Bu tasarım deseni, bir veri grubunun (liste, ağaç, yığın vb.) iç yapısını dışarıya açmadan, o



Şekil 17.15: Yineleyici Deseni UML Diyagramı

grubun elemanları üzerinde sırayla gezinebilmemizi sağlayan davranışsal bir desendir. Bu desen sayesinde, verilerin bellekte nasıl tutulduğunu (bir dizi mi yoksa birbirine bağlı halkalar mı?) bilmenize gerek kalmadan standart bir şekilde tüm elemanları dolaşabilirsiniz.

- Yineleyici (**Iterator**), elemanlar üzerinde gezinmek için gerekli olan standart arayüzü tanımlar. Genellikle **next()** (sonraki eleman), **hasNext()** (başka eleman var mı?) ve **current()** (mevcut eleman) gibi yöntemleri barındırır.
- Somut Yineleyici (**ConcreteIterator**), belirli bir veri yapısı (örneğin bir Vektör veya Liste) için gezinme mantığını uygular. O an hangi elemanda olduğunu (indis veya işaretçi bazlı) takip eder.
- Soyut Koleksiyon (**Aggregate**), koleksiyonun (**container**) kendisi için ortak bir arayüz sağlar. Kendi elemanlarını gezmek için kullanılacak olan uygun **Iterator** nesnesini oluşturan **createIterator()** adlı bir yöntemle sahiptir.
- Somut Koleksiyon (**ConcreteAggregate**), verileri asıl saklayan sınıftır. İstendiğinde, kendi iç yapısını en verimli şekilde gezecek olan somut yineleyiciyi (**ConcreteIterator**) geri döndürür.

Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <vector>
#include <memory> // std::unique_ptr için

//1. Soyut Iterator Arayüzü
class Iterator {
public:
    virtual ~Iterator() = default;
    virtual double first() = 0;
    virtual double next() = 0;
    virtual bool hasNext() const = 0; // isDone yerine daha yaygın isimlendirme
    virtual double currentItem() const = 0;
};

//2. Soyut Koleksiyon (Aggregate) Arayüzü
class Aggregate {
public:
    virtual ~Aggregate() = default;
```

```

    virtual void addItem(double item) = 0;
    virtual size_t size() const = 0;
    virtual double getItem(size_t index) const = 0;
    virtual std::unique_ptr<Iterator> createIterator() = 0;
};

//3. Somut Iterator (Concrete Iterator)
class ConcreteIterator : public Iterator {
private:
    Aggregate* aggregate; // Koleksiyona sadece referans/işaretçi tutar, mülkiyet almaz
    size_t current = 0;
public:
    explicit ConcreteIterator(Aggregate* pAggregate) : aggregate(pAggregate) {}
    double first() override {
        current = 0;
        return currentItem();
    }
    double next() override {
        if (hasNext()) {
            return aggregate->getItem(current++);
        }
        return -1.0; // Veya hata fırlatılabilir
    }
    bool hasNext() const override {
        return current < aggregate->size();
    }
    double currentItem() const override {
        return aggregate->getItem(current);
    }
};

//4. Somut Koleksiyon (Concrete Aggregate)
class ConcreteAggregate : public Aggregate {
private:
    std::vector<double> items;
public:
    void addItem(double item) override {
        items.push_back(item);
    }
    size_t size() const override {
        return items.size();
    }
    double getItem(size_t index) const override {
        return items.at(index); // out_of_range kontrolü için .at() daha güvenlidir
    }
    std::unique_ptr<Iterator> createIterator() override {
        // factory method: mülkiyeti istemciye (unique_ptr ile) devreder
        return std::make_unique<ConcreteIterator>(this);
    }
};

//5. Client (İstemci)
int main() { // Client (İstemci)
    auto aggregate = std::make_unique<ConcreteAggregate>();
    aggregate->addItem(10.5);
    aggregate->addItem(20.7);
    aggregate->addItem(30.9);
    auto iterator = aggregate->createIterator();
    std::cout << "Koleksiyon elemanlari geziliyor:" << std::endl;
    // Standart döngü yapısı
    for (iterator->first(); iterator->hasNext(); ) {
        std::cout << "Deger: " << iterator->next() << std::endl;
    }
}

```

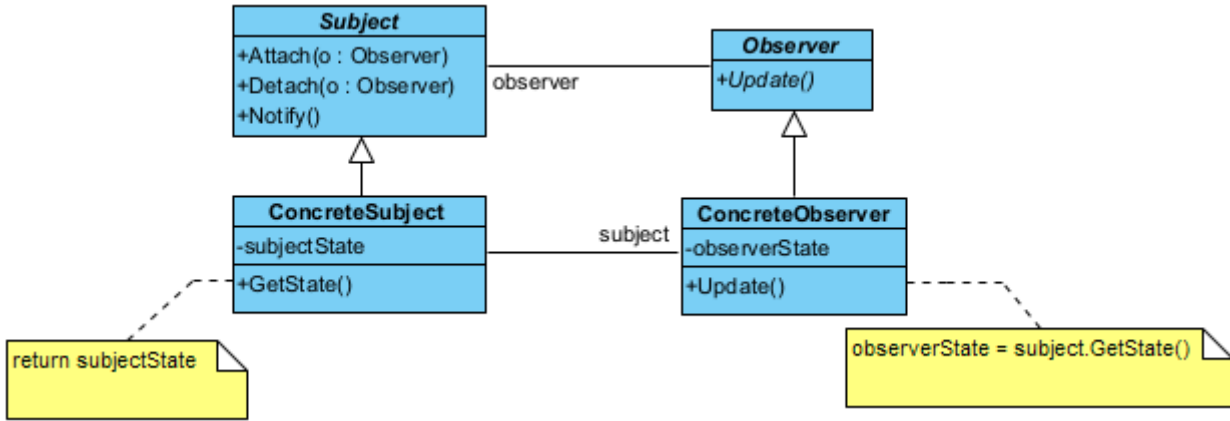
```

}
// Manuel delete gerekmez, her şey otomatik temizlenir
}
/*Program Çıktısı:
Koleksiyon elemanlari geziliyor:
Deger: 10.5
Deger: 20.7
Deger: 30.9
*/

```

§17.4.4 Gözlemci

Gözlemci deseni (observer pattern), sistemdeki diğer nesnelerin durum değişiklikleri hakkında bir veya daha fazla nesnenin bilgilendirilmesini sağlar.



Şekil 17.16: Gözlemci Deseni UML Diyagramı

Bu tasarım deseni, nesneler arasında bire-çok (**one-to-many**) bir bağımlılık kurmak için kullanılır. Bir nesnenin durumu değiştiğinde, ona bağlı olan tüm diğer nesnelerin (abonelerin) bu değişiklikten otomatik olarak haberdar edilmesini ve güncellenmesini sağlar. Bu desen, modern yazılımlardaki "olay güdümlü" (event-driven) sistemlerin ve MVC (**Model-View-Controller**) mimarisinin kalbidir.

- ◊ Bağlam (**Subject**), takip edilen asıl yayıncı nesnedir. Durumu değiştiğinde etrafa haber salar. İçinde bir "Gözlemci Listesi" tutar. Yeni gözlemci eklemek (**attach()**), çıkarmak (**detach()**) ve hepsine haber vermek (**notify()**) için yöntemler barındırır.
- ◊ Gözlemci (**Observer**), güncelleme mesajlarını alabilmek için gerekli olan standart abone arayüzünü tanımlar. Genellikle sadece tek bir **update()** metodu içerir.
- ◊ Somut Yayıncı (**ConcreteSubject**), gerçek veriyi saklayan sınıftır (Örnek olarak bir borsa takip sistemi veya haber sitesi). Kendi içindeki veri değiştiğinde, miras aldığı **notify()** metodunu çağırarak tüm aboneleri tetikler.
- ◊ Somut Gözlemci (**ConcreteObserver**), haberi alan ve buna göre tepki veren sınıftır. **update()** yöntemi tetiklendiğinde, yayıncıdan gelen yeni veriyi alır ve kendi içinde işler (Ekranı yazar, veritabanına kaydeder vb.).

Şimdi desenimizi kodlayalım;

```

//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <vector>
#include <string>
#include <memory>
#include <algorithm>

//1. Gözlemci: Observer Arayüzü
class Observer {
public:
    virtual ~Observer() = default; // Kritik: Sanal yıkıcı
    virtual void update(int state) = 0; // Durumu doğrudan parametre olarak alabilir (Push model)
};

//2. Bağlam/Yayıncı: Taban Sınıfı

```

```

class Subject {
private:
    // weak_ptr kullanımı: Gözlemci silinmişse Subject onu zorla hayatta tutmaz.
    std::vector<std::weak_ptr<Observer>> observers;
public:
    virtual ~Subject() = default;
    void attach(std::shared_ptr<Observer> observer) {
        observers.push_back(observer);
    }
    void detach(std::shared_ptr<Observer> observer) {
        observers.erase(std::remove_if(observers.begin(), observers.end(),
            [&observer](const std::weak_ptr<Observer>& wp) {
                return wp.expired() || wp.lock() == observer;
            }), observers.end());
    }
    void notify(int state) {
        for (auto it = observers.begin(); it != observers.end(); ) {
            if (auto obj = it->lock()) { // Nesne hala hayattaysa
                obj->update(state);
                ++it;
            } else { // Nesne silinmişse listeden temizle
                it = observers.erase(it);
            }
        }
    }
};

//3. Somut Bağlam/Yayıncı (Concrete Subject)
class NewsAgency : public Subject {
private:
    int newsId = 0;
public:
    void setNews(int id) {
        newsId = id;
        std::cout << "Haber Ajansi: Yeni haber yayınlandı (ID: " << id << ")\n";
        notify(newsId); // Otomatik bildirim
    }
};

//4. Somut Gözlemci (Concrete Observer)
class NewsChannel : public Observer {
private:
    std::string name;
public:
    explicit NewsChannel(std::string n) : name(std::move(n)) {}
    void update(int state) override {
        std::cout << " -> " << name << " kanalı bilgiyi aldı. Yeni haber ID: " << state <<
            "\n";
    }
};

//5. Client (İstemci)
int main() { // İstemci (Client)
    // Nesneleri paylaşımlı akıllı işaretçiler ile oluşturuyoruz
    auto agency = std::make_shared<NewsAgency>();
    auto channel1 = std::make_shared<NewsChannel>("TRT Haber");
    auto channel2 = std::make_shared<NewsChannel>("NTV");
    auto channel3 = std::make_shared<NewsChannel>("CNN Türk");
    agency->attach(channel1);
    agency->attach(channel2);
    agency->attach(channel3);
    // Bir durumu değiştirelim, bildirimler otomatik gidecek
}

```



```

agency->setNews(101);
std::cout << "-----" << std::endl;
agency->detach(channel2); // NTV ayrılıyor
agency->setNews(102);
// Manuel 'delete' yok, bellek sızıntısı imkansız.
}
/*Program Çıktısı:
Haber Ajansi: Yeni haber yayinlandi (ID: 101)
-> TRT Haber kanali bilgiyi aldi. Yeni haber ID: 101
-> NTV kanali bilgiyi aldi. Yeni haber ID: 101
-> CNN Turk kanali bilgiyi aldi. Yeni haber ID: 101
-----
Haber Ajansi: Yeni haber yayinlandi (ID: 102)
-> TRT Haber kanali bilgiyi aldi. Yeni haber ID: 102
-> CNN Turk kanali bilgiyi aldi. Yeni haber ID: 102
*/

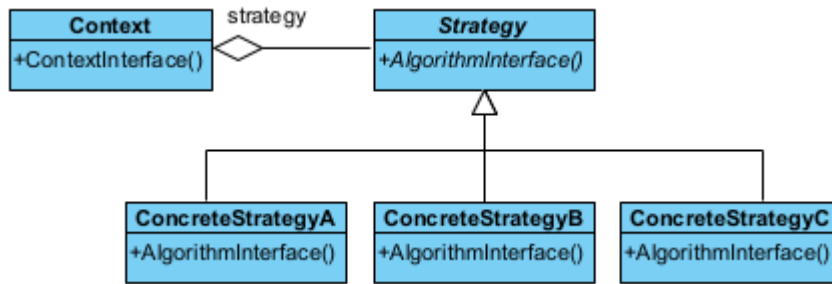
```

Bu desen aşağıdaki durumlarda kullanılır;

- ◊ Bir soyutlama (**abstraction**) sonucu ortaya çıkan bağımsız iki farklı yönün (**aspect**) ortaya çıktığı durumlarda, bu yönler birbirinden bağımsız olarak geliştirilebilir ve değiştirilebilir.
- ◊ Bir değişikliğin başka değişiklikleri gerektirdiği durumlar.
- ◊ Bir nesnenin, diğer nesnelerdeki değişiklikleri kendine vazife etmemesini sağlayan durumlar.

§17.4.5 Strateji

Strateji deseni (**strategy pattern**), belirli bir davranışı gerçekleştirmek için değiştirilebilen kapsüllenmiş (**encapsulated**) algoritmalar kümesini tanımlar.



Şekil 17.17: Strateji Deseni UML Diyagramı

Bu desenin görevi, bir işi yapmanın birden fazla yolu varsa, bu yolları (algoritmaları) sınıflara ayırır ve birbirinin yerine kullanılabilir hale getirir.

- ◊ Bağlam (**Context**), işi yapacak olan sınıf. Ancak işi "nasıl" yapacağını bir strateji nesnesine sorar.
- ◊ Strateji (**Strategy**): Tüm algoritmaların uyması gereken arayüzdür.
- ◊ Somut Strateji (**ConcreteStrategy**), farklı algoritmalar ile tanımlı stratejilerdir (Örnek olarak "Hızlı Sıralama", "Kabarcık Sıralama" veya "Kredi Kartıyla Öde", "Nakit Öde").

Şimdi desenimizi kodlayalım;

```

//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <memory>
#include <vector>

//1. Strateji Arayüzü
class Strategy {
public:
    virtual ~Strategy() = default; // Kritik: Sanal yıkıcı
    virtual void execute() const = 0;
};

//2. Somut Stratejiler
class ConcreteStrategyA : public Strategy {
public:
    void execute() const override {

```

```

        std::cout << "Algoritma A: Hizli ama daha fazla bellek kullaniyor.\n";
    }
};
class ConcreteStrategyB : public Strategy {
public:
    void execute() const override {
        std::cout << "Algoritma B: Yavas ama bellek dostu.\n";
    }
};
class ConcreteStrategyC : public Strategy {
public:
    void execute() const override {
        std::cout << "Algoritma C: Dengeli bir yaklasim.\n";
    }
};

//3. Baglam (Context)
class Context {
private:
    // Stratejinin mulkiyetini almaz, sadece referans/isaretcici tutar (Aggregation)
    Strategy* strategy_ = nullptr;
public:
    explicit Context(Strategy* strategy) : strategy_(strategy) {}
    void setStrategy(Strategy* strategy) {
        strategy_ = strategy;
    }
    void run() const {
        if (strategy_) {
            strategy_>execute();
        } else {
            std::cout << "Hata: Henuz bir strateji belirlenmedi!\n";
        }
    }
};

//4. Client (İstemci)
int main() {
    // Stratejileri akilli isaretcilerle yönetiyoruz
    auto s1 = std::make_unique<ConcreteStrategyA>();
    auto s2 = std::make_unique<ConcreteStrategyB>();
    auto s3 = std::make_unique<ConcreteStrategyC>();
    // Baglam (Context) nesnesini olustur ve baslangic stratejisini ata
    Context context(s1.get());
    std::cout << "--- Durum 1 ---" << std::endl;
    context.run();
    // Calisma zamaninda stratejiyi degistir
    context.setStrategy(s2.get());
    std::cout << "--- Durum 2 ---" << std::endl;
    context.run();
    context.setStrategy(s3.get());
    std::cout << "--- Durum 3 ---" << std::endl;
    context.run();
    // Manuel 'delete' yok, s1-s2-s3 otomatik temizlenir.
}

/*Program Çıktısı:
--- Durum 1 ---
Algoritma A: Hizli ama daha fazla bellek kullaniyor.
--- Durum 2 ---
Algoritma B: Yavas ama bellek dostu.
--- Durum 3 ---
Algoritma C: Dengeli bir yaklasim.
*/

```

Bu desenin **bağlam** nesnesinin, diğer tasarım desenlerinin biri ile iç içe kullanılması halinde, bu desen **sıfır desen** (**null pattern**) olarak adlandırılır. Sıfır desende, **Context** nesnesi akıllıdır ve her zaman ne yapacağını bilir. Bu desen aşağıdaki durumda kullanılır;

- ◊ Aralarında ilişki bulunan birçok sınıfın davranışlara bağlı olarak anlaşmazlığa düşmesini önlemek için,
- ◊ Birden çok algoritmanın olması durumunda,
- ◊ Algoritma, istemcinin bilmeyeceği bir veri yapısına sahip olduğu durumlarda,
- ◊ Bir sınıf kendi yöntemlerinde çok fazla sayıda duruma göre seçim (**conditional choice**) kullanıyorsa.

Aşağıda bu desene ilişkin sıralama algoritmalarının seçildiği gerçek bir örnek verilmiştir;

```
//C++ 14 ve üzeri ile derlenmelidir.
#include <iostream>
#include <vector>
#include <memory>
#include <algorithm> // std::sort için

// --- Abstract Strategy ---
class SortStrategy {
public:
    virtual ~SortStrategy() = default;
    virtual void sort(std::vector<int>& data) = 0;
};

// --- Concrete Strategy A: Quick Sort ---
class QuickSort : public SortStrategy {
public:
    void sort(std::vector<int>& data) override {
        std::cout << "QuickSort algoritması kullaniliyor..." << std::endl;
        // Gerçek QuickSort yerine std::sort kullanalım
        // (genelde introsort/quicksort türevidir)
        std::sort(data.begin(), data.end());
    }
};

// --- Concrete Strategy B: Merge Sort ---
class MergeSort : public SortStrategy {
public:
    void sort(std::vector<int>& data) override {
        std::cout << "MergeSort algoritması kullaniliyor..." << std::endl;
        // Örnek amaçlı standart stable_sort (merge sort benzeri garantiler sunar)
        std::stable_sort(data.begin(), data.end());
    }
};

// --- Context: Sorter ---
class Sorter {
private:
    std::unique_ptr<SortStrategy> strategy_;
public:
    // Stratejiyi çalışma zamanında değiştirmemizi sağlar
    void setStrategy(std::unique_ptr<SortStrategy> strategy) {
        strategy_ = std::move(strategy);
    }
    void sort(std::vector<int>& data) {
        if (!strategy_) {
            std::cout << "Hata: Önce bir strateji belirlenmelidir!" << std::endl;
            return;
        }
        strategy_->sort(data);
    }
};

// Yardımcı fonksiyon: Verileri ekrana basmak için
void printVector(const std::vector<int>& data) {
    for (int n : data)
```

```

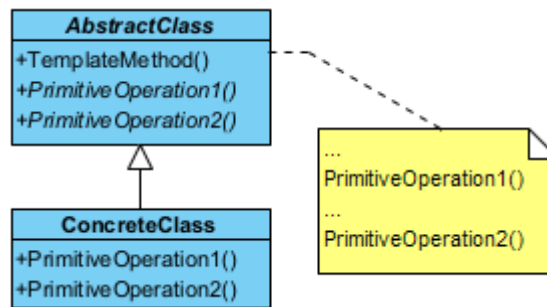
        std::cout << n << " ";
        std::cout << std::endl;
    }
    int main() {
        std::vector<int> sayilar = {34, 12, 5, 78, 1, 45};
        Sorter siralayici;
        std::cout << "Orijinal liste: ";
        printVector(sayilar);
        // 1. QuickSort Stratejisini Seçelim
        siralayici.setStrategy(std::make_unique<QuickSort>());
        siralayici.sort(sayilar);
        printVector(sayilar);
        // Listeyi tekrar karıştıralım
        sayilar = {99, 2, 56, 11, 0, 7};
        std::cout << "\nYeni liste: ";
        printVector(sayilar);
        // 2. MergeSort Stratejisine Geçelim (Çalışma zamanında değişim)
        siralayici.setStrategy(std::make_unique<MergeSort>());
        siralayici.sort(sayilar);
        printVector(sayilar);
    }
    /* Programın Çıktısı:
    Orijinal liste: 34 12 5 78 1 45
    QuickSort algoritması kullanılıyor...
    1 5 12 34 45 78

    Yeni liste: 99 2 56 11 0 7
    MergeSort algoritması kullanılıyor...
    0 2 7 11 56 99
    */

```

§17.4.6 Şablon Yöntem

Şablon yöntem deseninde (**template method pattern**), bir algoritmanın çerçevesini belirler ve uygulayıcı sınıfların gerçek davranışı tanımlamasına olanak tanır.



Şekil 17.18: Şablon Yöntem Deseni UML Diyagramı

Bu desen, nesne yönelimli programlamada "algoritma iskeleti" oluşturmak için kullanılır. Bir işin adımlarını (sırasını) belirler ancak bu adımların bazılarının nasıl yapılacağını alt sınıflara bırakır. Kod tekrarını önlemek ve bir işlemin temel yapısını korumak için mükemmeldir.

- ◇ Soyut Sınıf (**AbstractClass**), algoritmanın ana iskeletini ve Şablon yöntemi (**TemplateMethod()**) barındırır. Değişmez adımları kendi içinde tanımlar, değişebilecek adımları ise soyut (**virtual**) olarak işaretler. Şablon yöntem genellikle **final** olarak tanımlanır ki algoritmanın sırası bozulmasın.
- ◇ Somut Sınıf (**ConcreteClass**); Soyut sınıftaki "boşlukları" doldurur. Algoritmanın değişken adımlarını kendi ihtiyacına göre özelleştirir. Algoritmanın genel akışına müdahale edemez, sadece kendisine izin verilen adımları yazar.

Şimdi desenimizi kodlayalım;

```

//C++14 ve üzeri ile derlenmelidir.
#include <iostream>
#include <memory> // std::unique_ptr için

```

```
//1. Soyut Sınıf (Abstract Class)
class AbstractClass {
public:
    // Kritik: Polimorfik sınıflarda sanal yıkıcı şarttır.
    virtual ~AbstractClass() = default;
    // Şablon Yöntem: Algoritmanın iskeletini sabitler.
    // Alt sınıflar bu sırayı değiştiremez.
    void templateMethod() {
        baseOperation();           // Ortak adım
        requiredOperation1();       // Özelleştirilebilir adım
        hook();                     // İsteğe bağlı adım (Hook)
        requiredOperation2();       // Özelleştirilebilir adım
    }
protected:
    // Temel operasyon: Tüm alt sınıflar için aynıdır.
    void baseOperation() const {
        std::cout << "Sistem: Algoritmanın ortak temel adimi calistiriliyor.\n";
    }
    // Alt sınıfların mutlaka uygulaması gereken adımlar (Saf sanal)
    virtual void requiredOperation1() = 0;
    virtual void requiredOperation2() = 0;
    // Hook (Kanca): Alt sınıflar isterse ezer, istemezse boş kalır.
    virtual void hook() {}
};

//2. Somut Sınıf (Concrete Class)
class ConcreteClass : public AbstractClass {
protected:
    // override anahtar kelimesi okunabilirliği ve güvenliği artırır.
    void requiredOperation1() override {
        std::cout << "ConcreteClass: Adim 1 özel olarak uygulandi.\n";
    }
    void requiredOperation2() override {
        std::cout << "ConcreteClass: Adim 2 özel olarak uygulandi.\n";
    }
    // İsteğe bağlı: Hook'u ezebiliriz.
    void hook() override {
        std::cout << "ConcreteClass: Hook (Kanca) devreye girdi.\n";
    }
};

//3. Client (İstemci)
int main() {
    // Akıllı işaretçi kullanarak bellek yönetimini otomatige bağlıyoruz.
    std::unique_ptr<AbstractClass> algoritma = std::make_unique<ConcreteClass>();
    std::cout << "--- Algoritma Baslatiliyor ---\n";
    algoritma->templateMethod();
    // Kapsam dışına çıkınca bellek otomatik temizlenir.
}

/*Program Çıktısı:
--- Algoritma Baslatiliyor ---
Sistem: Algoritmanın ortak temel adimi calistiriliyor.
ConcreteClass: Adim 1 özel olarak uygulandi.
ConcreteClass: Hook (Kanca) devreye girdi.
ConcreteClass: Adim 2 özel olarak uygulandi.
*/
```

Bu desende, kullanılacak algoritmalarından hangilerinin standart, hangilerinin somut sınıflara özgü olacağı belirlenmelidir;

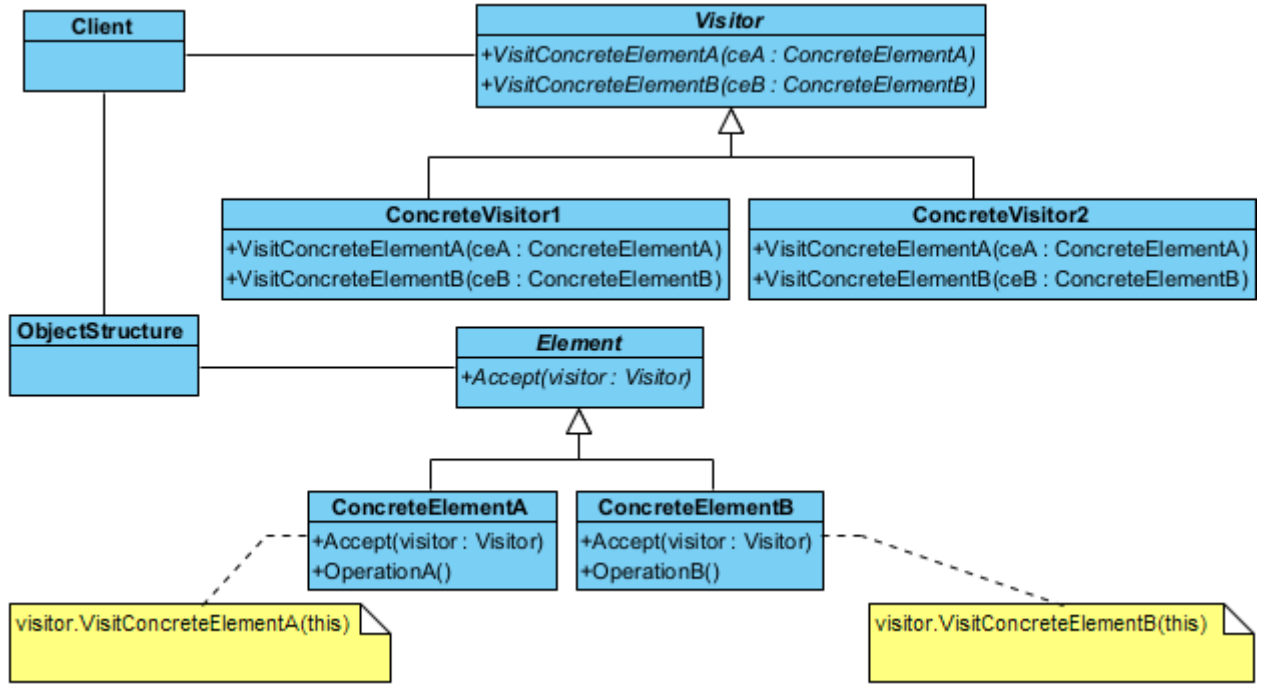
- ◊ İçinde “Bizi çağırmayın! Biz sizi çağıracağız.” sloganını taşıyan yöntemler olan bir soyut bir taban sınıf tasarlanmalıdır.
- ◊ Taban sınıf içinde algoritmanın kabuğunu oluşturacak standart adımları içeren şablon yöntem tanım-

lanır.

- ◊ Somut sınıflarda detaylandırılacak yöntemlere ilişkin sanal yöntemler taban sınıfta tanımlanır.
- ◊ Şablon yöntem içinde algoritmayı oluşturan yöntemler çağrılır.
- ◊ Şablon yöntem içinde yer almayan tüm detay kodlamalar somut sınıflar için de yapılır.

§17.4.7 Ziyaretçi

Ziyaretçi deseni (visitor pattern), bir nesne grubunun (koleksiyonun) yapısını değiştirmeden, bu nesneler üzerinde uygulanacak yeni işlemleri dışarıdan eklememizi sağlayan davranışsal bir desendir. Bu desen, "Veri Yapısı" ile "Algoritma"yı birbirinden ayırır. Eğer elinizde sabit bir sınıf hiyerarşisi varsa (örneğin bir dokümandaki paragraflar, resimler, tablolar) ve sürekli bu sınıflar üzerinde yeni işlemler (PDF'e dönüştür, HTML'e dönüştür, kelime say) tanımlamanız gerekiyorsa, her seferinde o sınıfların içine girip kod yazmak yerine ziyaretçi desenini kullanırsınız.



Şekil 17.19: Ziyaretçi Deseni UML Diyagramı

- ◊ Ziyaretçi (**Visitor**), koleksiyondaki her bir somut nesne türü için bir ziyaret yöntemi (**visit()**) tanımlayan soyut bir sınıftır. "Eğer bir Paragraf nesnesine gidersem ne yapacağım?", "Eğer bir Resim nesnesine gidersem ne yapacağım?" sorularının imzasını belirler.
- ◊ Somut Ziyaretçi (**ConcreteVisitor**), gerçek operasyonu gerçekleştirir. Algoritmanın kendisidir. Örneğin ExportToPDFVisitor sınıfı, her nesne türünün PDF'e nasıl dönüştürüleceğini bilir.
- ◊ Eleman (**Element**), bir ziyaretçiyi kabul edebilmek için standart bir "kapı" açan soyut bir sınıftır. Ziyaretçi kabul etmek için **accept(Visitor* v)** yöntemini barındırır.
- ◊ Somut Elemanlar (**ConcreteElement**), veri yapısının asıl parçalarıdır. **accept()** yöntemi çağrıldığında, ziyaretçiye "Ben bir [NesneTürü]'yüm, beni ziyaret et" der (**v->visit(this)**). Buna yazılım dünyasında **çifte sevkیات (double dispatch)** denir.

Şimdi desenimizi kodlayalım;

```
//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <vector>
#include <memory>
#include <string>
```

```
// 1. İleriye Dönük Bildirimler (Forward Declarations): Böyle sınıflar olacak!
```

```
class ConcreteElementA;
class ConcreteElementB;
```

```
// 2. Visitor (Ziyaretçi) Arayüzü
```

```
class Visitor {
public:
```

```

    virtual ~Visitor() = default; // Kritik: Sanal yıkıcı
    virtual void visit(ConcreteElementA* element) = 0;
    virtual void visit(ConcreteElementB* element) = 0;
};

// 3. Element (Eleman) Arayüzü
class Element {
public:
    virtual ~Element() = default;
    // "Double Dispatch" mekanizmasının kalbi: accept metodu
    virtual void accept(Visitor* visitor) = 0;
};

// 4. Somut Elemanlar
class ConcreteElementA : public Element {
public:
    void accept(Visitor* visitor) override {
        visitor->visit(this);
    }
    void operationA() const {
        std::cout << "ConcreteElementA: Özel işlem çalıştırılıyor.\n";
    }
};

class ConcreteElementB : public Element {
public:
    void accept(Visitor* visitor) override {
        visitor->visit(this);
    }
    void operationB() const {
        std::cout << "ConcreteElementB: Özel işlem çalıştırılıyor.\n";
    }
};

// 5. Somut Ziyaretçiler
class ConcreteVisitor1 : public Visitor {
public:
    void visit(ConcreteElementA* element) override {
        std::cout << "Visitor 1: ElementA ziyaret edildi.\n";
        element->operationA();
    }
    void visit(ConcreteElementB* element) override {
        std::cout << "Visitor 1: ElementB ziyaret edildi.\n";
        element->operationB();
    }
};

class ConcreteVisitor2 : public Visitor {
public:
    void visit(ConcreteElementA* element) override {
        std::cout << "Visitor 2: ElementA üzerinde farklı bir işlem yapılıyor.\n";
    }
    void visit(ConcreteElementB* element) override {
        std::cout << "Visitor 2: ElementB üzerinde farklı bir işlem yapılıyor.\n";
    }
};

// 6. Nesne Yapısı (Object Structure)
class ObjectStructure {
private:
    std::vector<std::unique_ptr<Element>> elements;
public:
    void add(std::unique_ptr<Element> e) {
        elements.push_back(std::move(e));
    }
};

```



```

    }
    void accept(Visitor* v) {
        for (const auto& e : elements) {
            e->accept(v);
        }
    }
};

// 7. Client (İstemci)
int main() {
    ObjectStructure structure;
    // Elemanları ekliyoruz (Sahiplik structure nesnesine geçiyor)
    structure.add(std::make_unique<ConcreteElementA>());
    structure.add(std::make_unique<ConcreteElementB>());
    // Ziyaretçileri oluşturuyoruz
    auto v1 = std::make_unique<ConcreteVisitor1>();
    auto v2 = std::make_unique<ConcreteVisitor2>();
    std::cout << "--- Ziyaretci 1 Basliyor ---\n";
    structure.accept(v1.get());
    std::cout << "--- Ziyaretci 2 Basliyor ---\n";
    structure.accept(v2.get());
    // Her şey otomatik ve güvenli bir şekilde silinir.
}

/*Program Çıktısı:
--- Ziyaretci 1 Basliyor ---
Visitor 1: ElementA ziyaret edildi.
ConcreteElementA: Özel işlem calistiriliyor.
Visitor 1: ElementB ziyaret edildi.
ConcreteElementB: Özel işlem calistiriliyor.
--- Ziyaretci 2 Basliyor ---
Visitor 2: ElementA üzerinde farkli bir işlem yapiliyor.
Visitor 2: ElementB üzerinde farkli bir işlem yapiliyor.
*/

```

Bu desen aşağıdaki durumlarda kullanılır;

- ◆ Nesneler birden fazla olup birçok arayüze sahip olduğunda, bu nesnelerin somut sınıfları ile bir işlem yapmak gerektiğinde,
- ◆ Sınıflar üzerinde yapılacak işlemlerin, sınıflar üzerinde bir iz bırakmamasını sağlamak amacıyla,
- ◆ Nadir değişen sınıf yapılarına, sınıf yapısını değiştirmeden, yeni işlemler eklemek gerektiğinde.

Bu deseni kullanmanın aşağıda sıralanan üstünlükleri sağlar;

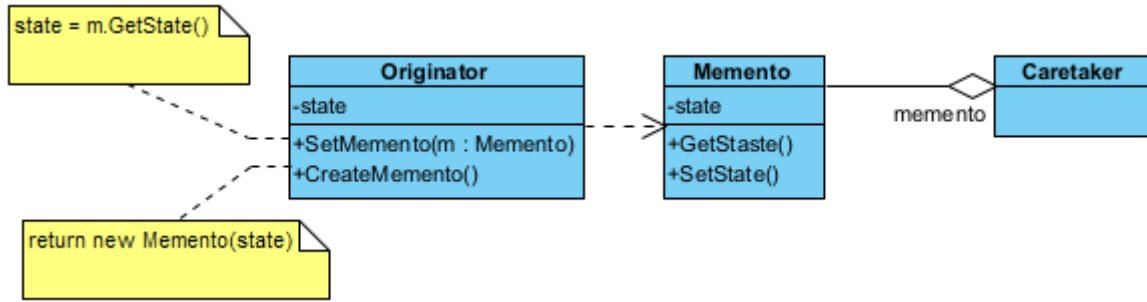
- ◆ Yeni işlemleri (**operation**) sisteme eklemeyi kolaylaştırır.
- ◆ Birbiriyle ilişkili olan işlemleri ilişkili olmayandan ayırmaya yardımcı olur.
- ◆ Yeni **ConcreteElement** nesnelerinin eklenmesini zorlaştırır.
- ◆ Bütün nesne hiyerarşisi baştanbaşa ziyaret edilebilir. Sınıflardaki tüm durumlar (**state**) biriktirilebilir.
- ◆ Bazı durumlarda kapsülleme (**encapsulation**) işlemini engeller. Yani nesnenin iç durumlarından hareketle herkese açık işlemler tanımlanabilir.

§17.4.8 Hatıra

Hatıra deseni (**memento pattern**), nesnelerin bir andaki durumlarını (**state**) saklayıp bir başka zaman da bu duruma geri dönmek gerektiğinde kullanılır. Bir nesnenin iç durumunun yakalanmasına ve dışarıda saklanmasına olanak tanır, böylece daha sonra geri yüklenebilir fakat tüm bunlar **kılıflamayı** (**encapsulation**) ihlal etmeden yapılır.

Hatıra deseninin amacı, nesnenin iç durumunu (**state**) bozmadan saklamak ve daha sonra geri yüklemektir.

- ◆ Yaratıcı (**Originator**), durumu değişen asıl nesnedir. Kendi "anlık görüntüsünü" (**snapshot**) oluşturur ve geri yükler.
- ◆ Hatıra (**Memento**), sadece veriyi tutan pasif nesnedir. Dışarıdan değiştirilemez; sadece **Originator** içeriğini okuyabilir.
- ◆ Bakıcı (**Caretaker**), hatıraları güvenli bir yerde (liste veya yığın) saklar. İçeriği asla değiştirmez, sadece saklar ve geri verir.



Şekil 17.20: Hatıra Deseni UML Diyagramı

Şimdi desenimizi kodlayalım;

```

#include <iostream>
#include <memory>
#include <string>
#include <vector>

//1. Memento: Durumu saklayan pasif nesne
class Memento {
private:
    // Sadece Originator bu veriye erişmeli (Encapsulation)
    friend class Originator;
    int state;
    explicit Memento(int pState) : state(pState) {}
    int getState() const { return state; }
};

// 2. Originator: Durumu değişen ve "hatıra" üreten asıl nesne
class Originator {
private:
    int state;
public:
    void setState(int pState) {
        state = pState;
        std::cout << "Originator: Durum degistirildi -> " << state << std::endl;
    }
    // Mevcut durumu bir Memento nesnesine paketler
    std::unique_ptr<Memento> save() {
        std::cout << "Originator: Durum yedekleniyor... (" << state << ")" << std::endl;
        return std::unique_ptr<Memento>(new Memento(state));
    }
    // Bir Memento nesnesinden durumu geri yukler
    void restore(const Memento* memento) {
        if (memento) {
            state = memento->getState();
            std::cout << "Originator: Durum geri yuklendi -> " << state << std::endl;
        }
    }
};

// 3. Caretaker: Hatıraları saklayan bekçi
class Caretaker {
private:
    std::vector<std::unique_ptr<Memento>> history;
    Originator& originator;
public:
    explicit Caretaker(Originator& o) : originator(o) {}
    void backup() {
        history.push_back(originator.save());
    }
}

```

```

    void undo() {
        if (history.empty()) return;
        auto memento = std::move(history.back());
        history.pop_back();
        originator.restore(memento.get());
    }
};

// 4. İstemci (Client)
int main() {
    Originator player;
    Caretaker saveManager(player);
    player.setState(100); // Bölüm 1
    saveManager.backup(); // Kayıt noktası
    player.setState(50);  // Can azaldı
    player.setState(0);   // Öldü!
    std::cout << "--- Geri Yukleniyor ---" << std::endl;
    saveManager.undo();   // Bölüm 1'e geri dön
}

/*Program Çıktısı:
Originator: Durum degistirildi -> 100
Originator: Durum yedekleniyor... (100)
Originator: Durum degistirildi -> 50
Originator: Durum degistirildi -> 0
--- Geri Yukleniyor ---
Originator: Durum geri yuklendi -> 100
*/

```

Aşağıdaki örnekte bir metin editörünün geçmiş özelliği modellenmiştir;

```

//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <vector>
#include <memory>

// --- Memento ---
// Sadece saklanacak veriyi tutar.
class EditorMemento {
private:
    std::string content;
public:
    EditorMemento(std::string text) : content(text) {}
    std::string getSavedContent() const { return content; }
};

// --- Originator ---
// Durumu kaydedilecek olan asıl nesne.
class TextEditor {
private:
    std::string text;
public:
    void type(std::string newText) { text += newText; }
    std::string getText() const { return text; }
    // Mevcut durumu bir Memento içine paketler.
    std::unique_ptr<EditorMemento> save() {
        return std::make_unique<EditorMemento>(text);
    }
    // Bir Memento'dan durumu geri yükler.
    void restore(const EditorMemento& memento) {
        text = memento.getSavedContent();
    }
};

```

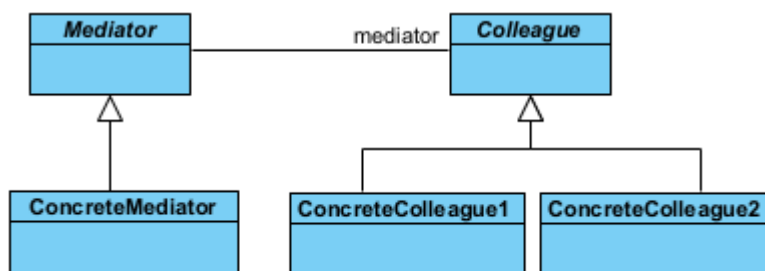
```
// --- Caretaker ---
// Geçmiş (Memento listesini) yönetir.
class History {
private:
    std::vector<std::unique_ptr<EditorMemento>> mementos;
public:
    void push(std::unique_ptr<EditorMemento> m) {
        mementos.push_back(std::move(m));
    }
    std::unique_ptr<EditorMemento> pop() {
        if (mementos.empty()) return nullptr;
        auto m = std::move(mementos.back());
        mementos.pop_back();
        return m;
    }
};

int main() {
    TextEditor editor;
    History history;
    // 1. Yazmaya başla ve ilk durumu kaydet
    editor.type("Merhaba ");
    history.push(editor.save());
    std::cout << "Güncel Metin: " << editor.getText() << std::endl;
    // 2. Yazmaya devam et ve ikinci durumu kaydet
    editor.type("Dunya!");
    history.push(editor.save());
    std::cout << "Güncel Metin: " << editor.getText() << std::endl;
    // 3. Bir hata yapalım
    editor.type(" Yanlis bir ekleme.");
    std::cout << "Hata Sonrasi: " << editor.getText() << std::endl;
    // 4. Geri al (Undo)
    auto undo = history.pop(); // Son kaydedilen "Merhaba Dunya!" idi, onu alalım.
    auto redo = history.pop(); // Bir önceki "Merhaba " idi.
    if (redo) {
        editor.restore(*redo);
        std::cout << "Geri Alindi: " << editor.getText() << std::endl;
    }
}

/*Program Çıktısı:
Güncel Metin: Merhaba
Güncel Metin: Merhaba Dunya!
Hata Sonrasi: Merhaba Dunya! Yanlis bir ekleme.
Geri Alindi: Merhaba
*/
```

§17.4.9 Ara Bulucu

Ara Bulucu deseni (**mediator pattern**), sınıfların arasındaki iletişimi basitleştirir. Bir iletişim nesnesi tanımlanır, diğer nesneler bu nesne üzerinden birbirleriyle haberleşir. Diğer nesnelerin birbiriyle doğrudan iletişim kurmasına izin verilmez. Amaç, iletişimin kontrol altına alınmasıdır. Bu tasarım deseni, bir dizi nesne



Şekil 17.21: Ara Bulucu Deseni UML Diyagramı

arasındaki karmaşık iletişim ağını basitleştirmek için kullanılır. Nesnelerin birbirleriyle doğrudan konuşmasını engeller ve tüm iletişimi tek bir "merkezi aracı" üzerinden yürütmelerini sağlar. Bu desen, özellikle nesne sayısı arttığında ortaya çıkan "örümcek ağı" gibi karmaşık bağlantıları (**tight coupling**) çözmek için birebirdir.

- ◇ Soyut Aracı (**Mediator**), nesneler arasındaki iletişim protokolünü (arayüzünü) tanımlar. "Hangi olay gerçekleştiğinde kimlere haber verilecek?" sorusunun genel çerçevesini çizer.
- ◇ Somut Aracı (**ConcreteMediator**), iletişim trafiğini yöneten asıl "santral" binasıdır. tüm meslektaşları (**Colleague**) tanır ve aralarındaki koordinasyonu sağlar. Bir bileşenden mesaj geldiğinde, mantık çerçevesinde diğerlerine iletir.
- ◇ Meslektaş (**Colleague**) Sınıflar, kendi işlevini yerine getiren bağımsız sınıflardır. Diğer sınıfların varlığından haberdar değildirler. Sadece "Aracı"ya (**Mediator**) haber verirler. "Ben düğmeye bastım, gerisini aracı halletsin" mantığıyla çalışırlar.

Şimdi desenimizi kodlayalım;

```
#include <iostream>
#include <vector>
#include <string>
#include <memory>
#include <algorithm>

// 1. İleriye Dönük Bildirim: İleride böyle bir sınıf olacak!
class Colleague;

// 2. Mediator (Ara bulucu) Arayüzü
class Mediator {
public:
    virtual ~Mediator() = default;
    virtual void registerColleague(std::shared_ptr<Colleague> colleague) = 0;
    virtual void relayMessage(const std::string& message, Colleague* sender) = 0;
};

// 3. Colleague (Meslektaş) Taban Sınıfı
class Colleague {
protected:
    std::string name;
    Mediator* mediator; // Döngüsel bağımlılığı önlemek için ham işaretçi veya weak_ptr
public:
    Colleague(std::string pName, Mediator* pMediator) : name(std::move(pName)),
        ↪ mediator(pMediator) {}
    virtual ~Colleague() = default;
    std::string getName() const { return name; }
    void send(const std::string& message) {
        mediator->relayMessage(message, this);
    }
    void receive(const std::string& message, const std::string& senderName) {
        std::cout << "[" << name << "]" " << senderName
        << " kisisinden mesaj aldı: " << message << std::endl;
    }
};

// 4. Somut Ara bulucu (Concrete Mediator)
class ChatRoom : public Mediator {
private:
    std::vector<std::shared_ptr<Colleague>> colleagues;
public:
    void registerColleague(std::shared_ptr<Colleague> colleague) override {
        colleagues.push_back(colleague);
    }
    void relayMessage(const std::string& message, Colleague* sender) override {
        std::cout << "--- LOG: " << sender->getName() << " bir mesaj yayınladı ---" << std::endl;

        for (const auto& colleague : colleagues) {
```

```

        // Mesajı gönderen kişiye geri göndermiyoruz
        if (colleague.get() != sender) {
            colleague->receive(message, sender->getName());
        }
    }
};

// 5. Somut Meslektaş (Concrete Colleague)
class User : public Colleague {
public:
    using Colleague::Colleague; // Yapıcıyı miras al
};

// 6. İstemci (Client)
int main() {
    // Arabulucu nesnesi
    auto oda = std::make_shared<ChatRoom>();
    // Meslektaşlar (Kullanıcılar)
    auto ilhan = std::make_shared<User>("Ilhan", oda.get());
    auto mehmet = std::make_shared<User>("Mehmet", oda.get());
    auto ayse = std::make_shared<User>("Ayse", oda.get());
    // Kayıt işlemleri
    oda->registerColleague(ilhan);
    oda->registerColleague(mehmet);
    oda->registerColleague(ayse);
    // İletişim başlıyor
    ilhan->send("Selam millet!");
    mehmet->send("Merhabalar Ilhan.");
    ayse->send("Nasılsınız?");
    // Her şey otomatik temizlenir.
}

/*Program Çıktısı:
--- LOG: Ilhan bir mesaj yayınladı ---
[Mehmet] Ilhan kisisinden mesaj aldı: Selam millet!
[Ayse] Ilhan kisisinden mesaj aldı: Selam millet!
--- LOG: Mehmet bir mesaj yayınladı ---
[Ilhan] Mehmet kisisinden mesaj aldı: Merhabalar Ilhan.
[Ayse] Mehmet kisisinden mesaj aldı: Merhabalar Ilhan.
--- LOG: Ayse bir mesaj yayınladı ---
[Ilhan] Ayse kisisinden mesaj aldı: Nasılsınız?
[Mehmet] Ayse kisisinden mesaj aldı: Nasılsınız?
*/

```

Bu desende, sistemde yalnızca bir adet ara bulucu olacaksa soyut **Mediator** nesnesi tanımlanmayabilir. Ayrıca ara bulucu ile meslektaşlar arasındaki iletişim iki türlü yapılır;

- ◊ Birisinde bu iletişim gözlemci deseni (**observer pattern**) ile sağlanır. **Colleague** sınıfı gözlemci desenindeki bağlam **Subject** sınıfı gibi davranır.
- ◊ Diğerinde ise **Mediator** nesnesine, görüşecekleri meslektaşları belirterek doğrudan ileti gönderirler.

Aşağıda havaalanındaki kule ile uçaklar arasındaki iletişimi modelleyen bir örnek verilmiştir;

```

#include <iostream>
#include <string>
#include <vector>
#include <memory>

// Forward declaration
class Airplane;

// --- Mediator Arayüzü ---
class IAirTrafficControl {
public:

```

```

    virtual ~IAirTrafficControl() = default;
    virtual void sendMessage(const std::string& message, Airplane* originator) = 0;
    virtual void registerAirplane(Airplane* airplane) = 0;
};

// --- Colleague (Meslektaş/Bileşen) ---
class Airplane {
protected:
    IAirTrafficControl* mediator_;
    std::string callSign_;
public:
    Airplane(IAirTrafficControl* mediator, std::string callSign) : mediator_(mediator),
        ↪ callSign_(callSign) {}
    virtual ~Airplane() = default;
    virtual void send(const std::string& message) {
        std::cout << "[" << callSign_ << "] Kuleye mesaj gönderiyor: " << message << std::endl;
        mediator_>sendMessage(message, this);
    }
    virtual void receive(const std::string& message) {
        std::cout << "[" << callSign_ << "] Mesaj aldı: " << message << std::endl;
    }
    std::string getCallSign() const { return callSign_; }
};

// --- Concrete Mediator (Somut Aracı) ---
class AtcTower : public IAirTrafficControl {
private:
    std::vector<Airplane*> airplanes_;
public:
    void registerAirplane(Airplane* airplane) override {
        airplanes_.push_back(airplane);
    }
    void sendMessage(const std::string& message, Airplane* originator) override {
        for (auto airplane : airplanes_) {
            // Mesajı gönderen uçak hariç herkese ilet (veya kurala göre işle)
            if (airplane != originator) {
                airplane->receive("Kule'den " + originator->getCallSign() + " bilgisi: " +
                    ↪ message);
            }
        }
    }
};

int main() {
    // 1. Aracı (Kule) oluşturulur
    auto tower = std::make_unique<AtcTower>();
    // 2. Meslektaşlar (Uçaklar) oluşturulur ve kuleye bildirilir
    auto boeing = std::make_unique<Airplane>(tower.get(), "THY-123");
    auto airbus = std::make_unique<Airplane>(tower.get(), "PGS-456");
    auto cessna = std::make_unique<Airplane>(tower.get(), "OZEL-789");
    tower->registerAirplane(boeing.get());
    tower->registerAirplane(airbus.get());
    tower->registerAirplane(cessna.get());
    // 3. İletişim merkezi üzerinden yürütülür
    boeing->send("Pist 1'e inis izni istiyorum.");
    std::cout << "-----" << std::endl;
    cessna->send("Pistten cikis yapildi.");
}

/* Program Çıktısı:
[THY-123] Kuleye mesaj gönderiyor: Pist 1'e inis izni istiyorum.
[PGS-456] Mesaj aldı: Kule'den THY-123 bilgisi: Pist 1'e inis izni istiyorum.
[OZEL-789] Mesaj aldı: Kule'den THY-123 bilgisi: Pist 1'e inis izni istiyorum.
*/

```



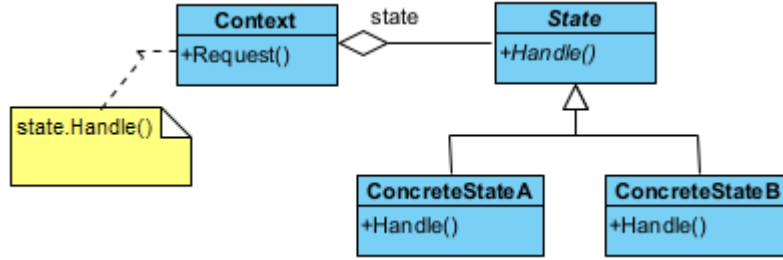
```

-----
[OZEL-789] Kuleye mesaj gonderiyor: Pistten cikis yapildi.
[THY-123] Mesaj aldi: Kule'den OZEL-789 bilgisi: Pistten cikis yapildi.
[PGS-456] Mesaj aldi: Kule'den OZEL-789 bilgisi: Pistten cikis yapildi.
*/

```

§17.4.10 Durum

Durum deseni (state pattern), nesnenin durumunu davranışına bağlar, nesnenin içsel durumuna göre farklı şekillerde davranmasına olanak tanır.



Şekil 17.22: Durum Deseni UML Diyagramı

Bu desenin amacı, nesnenin iç durumu değiştiğinde davranışını da değiştirmek. Nesne sanki sınıfını değiştiriyormuş gibi görünür.

- ◊ Bağlam (**Context**), istemcinin (**Client**) etkileşime girdiği ana sınıftır. Mevcut durum nesnesini bir referans olarak tutar.
- ◊ Soyut Durum (**State**), tüm somut durumların uyması gereken arayüzü (Interface) tanımlar.
- ◊ Somut Durumlar (**ConcreteState**), Her sınıf, belirli bir durumdaki davranışı temsil eder (Örn: Oynatılıyor, Duraklatıldı). Bir durumdan diğerine geçiş mantığını genellikle bu sınıflar yönetir.

Şimdi desenimizi kodlayalım;

```

//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <memory>

// 1. State Arayüzü (Soyut Durum)
class Context; // Forward declaration
class State {
public:
    virtual ~State() = default;
    virtual void handle(Context* context) = 0;
};

// 2. Context (Bağlam)
class Context {
private:
    std::unique_ptr<State> currentState;
public:
    explicit Context(std::unique_ptr<State> state) : currentState(std::move(state)) {}
    void transitionTo(std::unique_ptr<State> state) {
        std::cout << "Context: Durum degistiriliyor...\n";
        currentState = std::move(state);
    }
    void request() {
        currentState->handle(this);
    }
};

// 3. Somut Durumlar
class ConcreteStateB : public State {
public:
    void handle(Context* context) override; // İleride tanımlanacak
};

```

```

class ConcreteStateA : public State {
public:
    void handle(Context* context) override {
        std::cout << "ConcreteStateA: Talep isleniyor. B durumuna gecis tetikleniyor.\n";
        context->transitionTo(std::make_unique<ConcreteStateB>());
    }
};

void ConcreteStateB::handle(Context* context) {
    std::cout << "ConcreteStateB: Talep isleniyor. A durumuna geri donuluyor.\n";
    context->transitionTo(std::make_unique<ConcreteStateA>());
}

// 4. İstemci (Client)
int main() {
    // Başlangıç durumu A ile Context oluşturuluyor
    auto context = std::make_unique<Context>(std::make_unique<ConcreteStateA>());
    // İstekler gönderildikçe durumlar kendi içlerinde birbirini tetikler
    context->request(); // A -> B geçişi
    context->request(); // B -> A geçişi
    context->request(); // A -> B geçişi
    // Manuel delete gerekmez.
}

/*Program Çıktısı:
ConcreteStateA: Talep isleniyor. B durumuna gecis tetikleniyor.
Context: Durum degistiriliyor...
ConcreteStateB: Talep isleniyor. A durumuna geri donuluyor.
Context: Durum degistiriliyor...
ConcreteStateA: Talep isleniyor. B durumuna gecis tetikleniyor.
Context: Durum degistiriliyor...
*/

```

Bu desenin bağlam (**Context**) nesnesinin, diğer tasarım desenleri ile iç içe kullanılması halinde, bu desen, **sıfır desen** (**null pattern**) olarak adlandırılır. Ayrıca bu desenle ilgili olarak iki önemli şeyden bahsetmek gerekir:

- ♦ Sahip olunan duruma göre icra edilecek davranış, tanımlanan **State** sınıfına bağlıdır. Yani ilgili davranış somut sınıflardan biri tarafından icra edilir. Bu nedenle somut state (**ConcreteState**) sınıfları içinde tanımlanan yöntemler, durumu kullanan bağlam (**Context**) sınıf içinde tanımlanmak zorundadır.
- ♦ Durumu kullanan bağlam (**Context**) sınıf içinde hangi duruma geçildiğinin anlaşılması için ilgili duruma ilişkin tanımlanan özellik (**property**) içinde **set** yöntemi kullanılmak zorundadır.

Bu desen netice olarak, durum geçişlerini (**state transition**) açığa çıkarır. Bu deseni kullanırken aşağıda verilen durumlar özellikle irdelenmelidir;

- ♦ **Durum geçişleri kim tarafından tanımlanmalıdır?:** Bu desen, hangi şartlarda durumların ayrıştırılacağını belirtmez. Eğer şartlar belirgin ise bu işlem bağlam (**Context**) nesnesi tarafından yapılır. Bu şartlar belirgin değil ve karmaşık ise bu işlem **State** nesnesi ve halefi (**successor**) tarafından yapılır.
- ♦ **Tablo tabanlı alternatif:** Bu durumda durum geçişleri bir tablo aracılığı ile yapılır. Tablo üzerine sağlanacak her bir durum yazılır. Bu durumda, durum geçişlerini sağlayacak değişiklikler yeniden kodlama gerektirmez. Ancak bu yöntemde hız düşer. Durum geçişleri tekdüze (**uniform**) tanımlanmak zorundadır.
- ♦ **State nesnelerini yaratma ve yok etme:** Bu nesneler çalıştırma anında yaratılıp yok edildiklerinden, bu süre zarfında yapılacak işlemler için bir çözüm bulunmalıdır. İki yaklaşım vardır. Birincisi, State nesnesini ihtiyaç olduğunda yaratmak kullanmak ve öldürmektir. İkincisi ise bu nesneleri baştan yaratıp hiç öldürmemektir. Genellikle birinci yöntem tercih edilir.
- ♦ **Dinamik kalıtım (dynamic inheritance) kullanma:** Bazı durumlarda nesne, ait olduğu sınıfı çalıştırma anında değiştirir. Bu durumda talep edilen isteğin yerine getirilmesi tam olarak başarılabilir. Ancak birçok nesne yönelimli programlama dili bunu desteklemez.

Bu desen için en gerçekçi örneklerden biri **otomat makineleridir** (**vending machine**). Makinenin davranışı; içinde para olup olmasına, ürünün kalıp kalmamasına veya ürün seçilip seçilmediğine göre tamamen değişir. Eğer para yoksa "Ürün Seç" düğmesi çalışmaz; para varsa çalışır;

```

//C++14 ve üzeri ile derlenmelidir
#include <iostream>

```

```

#include <memory>
#include <string>

// --- Forward Declarations ---
class VendingMachine;

// --- Abstract State ---
class State {
public:
    virtual ~State() = default;
    virtual void insertMoney() = 0;
    virtual void ejectMoney() = 0;
    virtual void selectProduct() = 0;
    virtual void dispense() = 0;
};

// --- Concrete States ---
// 1. Para Yok Durumu
class NoMoneyState : public State {
    VendingMachine& machine;
public:
    NoMoneyState(VendingMachine& m) : machine(m) {}
    void insertMoney() override;
    void ejectMoney() override { std::cout << "Hata: Zaten para yok.\n"; }
    void selectProduct() override { std::cout << "Hata: Once para yuklemelisiniz.\n"; }
    void dispense() override { std::cout << "Hata: Odeme yapilmadan urun verilmez.\n"; }
};

// 2. Para Var Durumu
class HasMoneyState : public State {
    VendingMachine& machine;
public:
    HasMoneyState(VendingMachine& m) : machine(m) {}
    void insertMoney() override { std::cout << "Bilgi: Zaten para yuklu.\n"; }
    void ejectMoney() override;
    void selectProduct() override;
    void dispense() override { std::cout << "Bilgi: Once urun secimi yapmalisiniz.\n"; }
};

// 3. Ürün Satıldı/Teslimat Durumu
class SoldState : public State {
    VendingMachine& machine;
public:
    SoldState(VendingMachine& m) : machine(m) {}
    void insertMoney() override { std::cout << "Lutfen bekleyin: Urununuz hazirlaniliyor.\n"; }
    void ejectMoney() override { std::cout << "Hata: Islem geri alinamaz, urun veriliyor.\n"; }
    void selectProduct() override { std::cout << "Hata: Islem zaten devam ediyor.\n"; }
    void dispense() override;
};

// 4. Stok Yok Durumu
class SoldOutState : public State {
public:
    void insertMoney() override { std::cout << "Hata: Stok bitti, para yukleyemezsiniz.\n"; }
    void ejectMoney() override { std::cout << "Hata: Para yok.\n"; }
    void selectProduct() override { std::cout << "Hata: Stok bitti.\n"; }
    void dispense() override { std::cout << "Hata: Stok yok.\n"; }
};

// --- Context: VendingMachine ---
class VendingMachine {
private:

```

```

    std::unique_ptr<State> currentState;
    int count = 0;
public:
    VendingMachine(int productCount) : count(productCount) {
        if (count > 0)
            currentState = std::make_unique<NoMoneyState>(*this);
        else
            currentState = std::make_unique<SoldOutState>();
    }
    void setState(std::unique_ptr<State> state) {
        currentState = std::move(state);
    }
    // Proxy metodlar
    void insertMoney() { currentState->insertMoney(); }
    void ejectMoney() { currentState->ejectMoney(); }
    void selectProduct() { currentState->selectProduct(); }
    void dispense() { currentState->dispense(); }
    void releaseProduct() {
        if (count > 0) {
            std::cout << ">>> URUN VERILDI. Afiyet olsun!\n";
            count--;
        }
    }
    int getCount() const { return count; }
};

// --- Yöntem Tanımlamaları (Durum Geçiş Mantığı) ---
void NoMoneyState::insertMoney() {
    std::cout << "Para kabul edildi.\n";
    machine.setState(std::make_unique<HasMoneyState>(machine));
}
void HasMoneyState::ejectMoney() {
    std::cout << "Para iade ediliyor...\n";
    machine.setState(std::make_unique<NoMoneyState>(machine));
}
void HasMoneyState::selectProduct() {
    std::cout << "Urun secildi, hazirlaniyor...\n";
    machine.setState(std::make_unique<SoldState>(machine));
    machine.dispense(); // Otomatik teslimat tetiklemesi
}
void SoldState::dispense() {
    machine.releaseProduct();
    if (machine.getCount() > 0) {
        machine.setState(std::make_unique<NoMoneyState>(machine));
    } else {
        std::cout << "Dikkat: Son urun satildi!\n";
        machine.setState(std::make_unique<SoldOutState>());
    }
}

int main() {
    // 2 ürünlük bir otomat oluşturalım
    VendingMachine machine(2);
    std::cout << "---- Senaryo 1: Basarili Alim ---\n";
    machine.insertMoney();
    machine.selectProduct();
    std::cout << "---- Senaryo 2: Para Iadesi ---\n";
    machine.insertMoney();
    machine.ejectMoney();
    std::cout << "---- Senaryo 3: Son Urunu Alma ---\n";
    machine.insertMoney();
    machine.selectProduct();
}

```

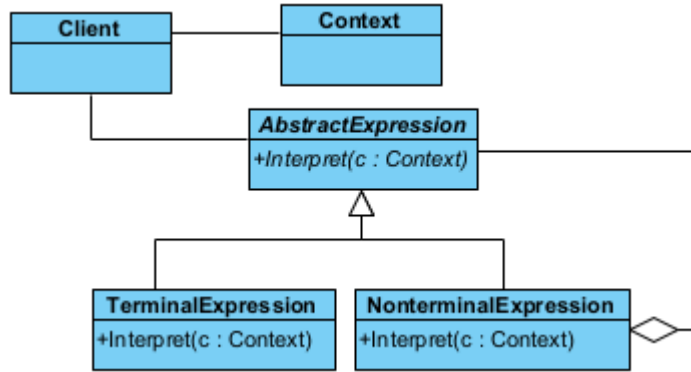
```

std::cout << "--- Senaryo 4: Stok Bittiginde Deneme ---\n";
machine.insertMoney();
}
/* Programın Çıktısı:
--- Senaryo 1: Basarili Alim ---
Para kabul edildi.
Urun secildi, hazirlaniyor...
*/

```

§17.4.11 Yorumlayıcı

Yorumlayıcı deseni (**interpreter pattern**), programlama dili yorumlama özelliğini uygulamalarımıza katmanın yolunu gösterir. Yani bir dil bilgisi için bir temsili ve aynı zamanda dil bilgisini anlayıp ona göre hareket etmeyi sağlayacak bir mekanizmayı tanımlar.



Şekil 17.23: Yorumlayıcı Deseni UML Diyagramı

Bu desenin amacı, belirli bir gramer yapısına sahip ifadeleri (matematiksel işlemler, SQL komutları vb.) çözümlenektir.

- ◊ Soyut İfade (**AbstractExpression**), tüm düğümlerin sahip olduğu **interpret()** yöntemini tanımlayan arayüzdür.
- ◊ Uç İfade (**TerminalExpression**), gramerin en küçük birimleridir (Sayılar veya değişkenler). Daha fazla parçalanamazlar.
- ◊ Uç Olmayan İfade (**NonTerminalExpression**), gramerin kurallarını (Toplama, Çarpma vb.) temsil eder. Genellikle içinde diğer ifadeleri (uç yada veya uç olmayan) barındırır.
- ◊ Bağlam (**Context**), yorumlama sırasında ihtiyaç duyulan küresel bilgileri (Değişken değerleri, sembol tabloları) taşır.

Şimdi desenimizi kodlayalım;

```

//C++14 ve üzeri ile derlenmelidir
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <map>

// 1. Context (Bağlam): Yorumlama sırasında kullanılan küresel bilgiler (Değişken değerleri vb.)
class Context {
private:
    std::map<std::string, int> variables;
public:
    void setVariable(const std::string& name, int value) { variables[name] = value; }
    int getVariable(const std::string& name) const { return variables.at(name); }
};

// 2. AbstractExpression (Soyut İfade)
class AbstractExpression {
public:
    virtual ~AbstractExpression() = default;
    virtual int interpret(const Context& context) const = 0;
};

```

```
// 3. TerminalExpression (Uç İfade): Sayılar veya değişkenler gibi atomik birimler
class NumberExpression : public AbstractExpression {
private:
    int number;
public:
    explicit NumberExpression(int n) : number(n) {}
    int interpret(const Context&) const override { return number; }
};

class VariableExpression : public AbstractExpression {
private:
    std::string name;
public:
    explicit VariableExpression(std::string n) : name(std::move(n)) {}
    int interpret(const Context& context) const override { return context.getVariable(name); }
};

// 4. NonterminalExpression (Uç Olmayan İfade): Operatörler (+, -, *) gibi karmaşık yapılar
class AddExpression : public AbstractExpression {
private:
    std::unique_ptr<AbstractExpression> left;
    std::unique_ptr<AbstractExpression> right;
public:
    AddExpression(std::unique_ptr<AbstractExpression> l, std::unique_ptr<AbstractExpression> r)
        : left(std::move(l)), right(std::move(r)) {}
    int interpret(const Context& context) const override {
        return left->interpret(context) + right->interpret(context);
    }
};

// 5. İstemci (Client)
int main() {
    Context context;
    context.setVariable("x", 10);
    context.setVariable("y", 5);
    // İfade: (x + y) + 20
    // Ağaç yapısı oluşturuluyor
    auto expression = std::make_unique<AddExpression>(
        std::make_unique<AddExpression>( std::make_unique<VariableExpression>("x"),
        → std::make_unique<VariableExpression>("y") ), std::make_unique<NumberExpression>(20) );
    std::cout << "Sonuc: " << expression->interpret(context) << std::endl;
    // unique_ptr sayesinde tüm ağaç otomatik temizlenir
}
/*Program Çıktısı:
Sonuc: 35
*/
```

Bu deseni uygularken aşağıda verilen durumlar irdelenmelidir;

- ♦ **Soyut söz dizimin oluşturulması:** Bu desen yorumlanacak dile ilişkin soyut bir söz dizimin nasıl olması gerektiğini açıklamaz. Tabloya dayalı, ağaç yapısı şeklinde el becerisi ile veya doğrudan istemci tarafından yapılabilir.
- ♦ **Yorumlayacak işlemin tanımlanması:** Eğer işlemler ortak bir yapıdaysa yeni bir `Expression` nesnesi tanımlanarak yapılabilir. Daha karmaşık durumlarda ziyaretçi deseni kullanılır. Programlama dillerinin çoğu soyut bir söz dizimine sahiptir ve yorumlanacak öğeler ayrı bir Visitor nesnesi tarafından yorumlanır.
- ♦ **Cümle sonlarını bitiren ortak semboller için sineksiklet deseni kullanılabilir:** Cümle sonlarındaki semboller genellikle bilgi saklamazlar. Yorumlamaya gerek duyulmazlar. Bu nedenle sineksiklet deseni (**flyweight pattern**) yorumlanacak yerler için bir eleme yapar.

Aşağıda bir metindeki ifadeyi hesaplayan bir uygulama örneği verilmiştir;

```
//C++14 ve üzeri ile derlenmelidir
```

```

#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <sstream>
#include <stack>

// --- Önceki Örnekteki Sınıflar (Aynen Kalıyor) ---
class Context {};
class Expression {
public:
    virtual ~Expression() = default;
    virtual int interpret(Context& context) const = 0;
};
class NumberExpression : public Expression {
    int number;
public:
    explicit NumberExpression(int n) : number(n) {}
    int interpret(Context&) const override { return number; }
};
class AdditionExpression : public Expression {
    std::unique_ptr<Expression> left, right;
public:
    AdditionExpression(std::unique_ptr<Expression> l, std::unique_ptr<Expression> r)
        : left(std::move(l)), right(std::move(r)) {}
    int interpret(Context& c) const override { return left->interpret(c) + right->interpret(c); }
};
class MultiplicationExpression : public Expression {
    std::unique_ptr<Expression> left, right;
public:
    MultiplicationExpression(std::unique_ptr<Expression> l, std::unique_ptr<Expression> r)
        : left(std::move(l)), right(std::move(r)) {}
    int interpret(Context& c) const override { return left->interpret(c) * right->interpret(c); }
};

// --- Dinamik Interpreter Sınıfı ---
class Interpreter {
public:
    int interpret(const std::string& rawExpression) {
        Context context;
        auto expressionTree = buildExpressionTree(rawExpression);
        return expressionTree->interpret(context);
    }
private:
    // String ifadeyi parçalara (tokens) ayıran yardımcı metod
    std::vector<std::string> tokenize(const std::string& str) {
        std::vector<std::string> tokens;
        std::stringstream ss(str);
        std::string temp;
        while (ss >> temp) tokens.push_back(temp);
        return tokens;
    }
    // Gerçek Dinamik Ağaç Oluşturucu (Shunting-yard benzeri basitleştirilmiş mantık)
    std::unique_ptr<Expression> buildExpressionTree(const std::string& expression) {
        auto tokens = tokenize(expression);
        // İşlem önceliği için iki yığın (stack)
        std::stack<std::unique_ptr<Expression>> nodes;
        std::stack<char> operators;
        auto applyOp = [&]() {
            auto right = std::move(nodes.top()); nodes.pop();
            auto left = std::move(nodes.top()); nodes.pop();
            char op = operators.top(); operators.pop();

```



```

    if (op == '+')
        nodes.push(std::make_unique<AdditionExpression>(std::move(left),
            ↪ std::move(right)));
    else if (op == '*')
        nodes.push(std::make_unique<MultiplicationExpression>(std::move(left),
            ↪ std::move(right)));
};

for (const std::string& token : tokens) {
    if (isdigit(token[0])) {
        nodes.push(std::make_unique<NumberExpression>(std::stoi(token)));
    } else {
        char currentOp = token[0];
        // İşlem önceliği kontrolü: Çarpma (+) dan önce yapılır
        while (!operators.empty() &&
            ((currentOp == '+' && (operators.top() == '+' || operators.top() == '*')) ||
            ↪ (currentOp == '*' && operators.top() == '*'))) {
            applyOp();
        }
        operators.push(currentOp);
    }
}

while (!operators.empty()) applyOp();
return std::move(nodes.top());
}

};

int main() {
    Interpreter parser;
    // Farklı kombinasyonları test edelim:
    std::string expr1 = "2 + 3 * 4"; // Beklenen: 14
    std::string expr2 = "10 * 2 + 5 * 3"; // Beklenen: 35
    std::string expr3 = "1 + 2 + 3 + 4"; // Beklenen: 10
    std::cout << expr1 << " = " << parser.interpret(expr1) << std::endl;
    std::cout << expr2 << " = " << parser.interpret(expr2) << std::endl;
    std::cout << expr3 << " = " << parser.interpret(expr3) << std::endl;
}

/*Program Çıktısı:
2 + 3 * 4 = 14
10 * 2 + 5 * 3 = 35
1 + 2 + 3 + 4 = 10
*/

```

17.5 MVC Mimari Deseni

MVC (**Model-View-Controller**), bir yazılım projesinde kodları görevlerine göre üç ana parçaya ayıran bir mimari desendir. Temel amacı, veriyi (mantık), kullanıcı arayüzünü ve bu ikisi arasındaki iletişimi birbirinden bağımsız hale getirerek kodun yönetimini kolaylaştırmaktır.

- ♦ **Veri ve Mantık (model)**: Uygulamanın beynidir. Veritabanı işlemleri, hesaplamalar ve kurallar burada döner. "Veri nedir?" sorusuna cevap verir.
- ♦ **Arayüz/Görünüm (view)**: Kullanıcının gördüğü ekrandır (HTML, CSS, butonlar). Sadece veriyi gösterir, arka planda ne olduğuyla ilgilenmez. "Veri nasıl görünmeli?" sorusuna cevap verir.
- ♦ **Aracı/Kontrolcü (controller)**: Kullanıcıdan gelen istekleri karşılar. **Model**'den veriyi alır ve hangi **View**'da gösterileceğine karar verir. "Hangi veri nereye gitmeli?" sorusuna cevap verir.

Basit bir restoran senaryosunda MVC'yi bir restorana benzetebiliriz;

- ♦ **Model (Aşçı)**: Mutfağın içindedir. Yemeği hazırlar, malzemeleri kontrol eder. Müşteriyi görmez, sadece siparişe göre yemeği (veriyi) üretir.
- ♦ **View (Yemek Masası/Sunum)**: Müşterinin önüne gelen tabak ve sunumdur. Müşteri yemeğin nasıl piştiğini görmez, sadece sunulan tabağı görür.
- ♦ **Controller (Garson)**: Müşteriden siparişi alır, aşçıya iletir. Yemek piştiğinde ise tabağı alır ve müşteriye servis eder.

Bir başka örnek olarak kullanıcı profili verilebilir. Diyelim ki bir kullanıcının profil bilgilerini ekrana

getireceksiniz:

- ◊ Kullanıcı (User): Profil sayfasına tıklar (istek/request).
- ◊ Controller: Bu isteği yakalar. "Kullanıcı verisi lazım" diyerek Model'e seslenir.
- ◊ Model: Veritabanına gider, "Ahmet, 25 yaşında, Ankara" bilgilerini bulur ve Controller'a geri verir.
- ◊ Controller: Aldığı bu "Ahmet" verisini alır ve "Profil_Ekrani.html" adlı View dosyasına gönderir.
- ◊ View: Gelen veriyi şık bir tasarıma yerleştirir ve kullanıcıya gösterir.

MVC kullanmalıyız çünkü;

- ◊ **Kod Karmaşasını Önler:** Tasarımcı sadece View ile, yazılımcı sadece Model/Controller ile ilgilenir.
- ◊ **Bakım Kolaylığı:** Görünümü değiştirmek istediğinizde veritabanı koduna dokunmanız gerekmez.
- ◊ **Yeniden Kullanılabilirlik:** Aynı Model verisini (örneğin kullanıcı listesi), hem bir web sayfasında hem de bir mobil uygulamada farklı View'lar ile gösterebilirsiniz.

C++ nesne yönelimli bir dil olduğu için MVC yapısını sınıflar arasındaki ilişkiyle kurgulamak, mantığı anlamak açısından çok faydalıdır. Bir "Öğrenci Bilgi Sistemi" üzerinden, bileşenlerin birbirine bağımlılığını (coupling) minimize edecek şekilde bir örnek verilebilir. Öğrenci Bilgi Sisteminde her bileşen kendi sorumluluğunu üstlenir: Model veriyi tutar, View ekrana basar, Controller ise akışı yönetir.

1. Model (Veri ve İş Mantığı): Sadece veriyi ve bu veriye erişim yöntemlerini içerir. Görselleştirme ile hiç ilgilenmez.

```
#include <string>
class OgrenciModel {
private:
    std::string isim;
    std::string numara;
public:
    OgrenciModel(std::string i, std::string n) : isim(i), numara(n) {}
    // Getter ve Setter yöntemleri
    void setIsim(std::string i) { isim = i; }
    std::string getIsim() const { return isim; }
    void setNumara(std::string n) { numara = n; }
    std::string getNumara() const { return numara; }
};
```

2. View (Görünüm): Verinin kullanıcıya nasıl sunulacağını belirler. Genellikle sadece "Yazdır" veya "Göster" gibi yöntemleri vardır.

```
#include <iostream>
class OgrenciView {
public:
    void ogrenciDetaylariniBas(std::string isim, std::string numara) {
        std::cout << "--- Ogrenci Bilgileri ---" << std::endl;
        std::cout << "Ad: " << isim << std::endl;
        std::cout << "No: " << numara << std::endl;
        std::cout << "-----" << std::endl;
    }
};
```

3. Controller (Denetleyici): Model ve View arasındaki köprüdür. Kullanıcıdan gelen güncellemeleri Model'e yansıtır ve View'u güncellenmeye zorlar.

```
class OgrenciController {
private:
    OgrenciModel& model;
    OgrenciView& view;
public:
    OgrenciController(OgrenciModel& m, OgrenciView& v) : model(m), view(v) {}
    void setOgrenciIsim(std::string i) { model.setIsim(i); }
    std::string getOgrenciIsim() { return model.getIsim(); }
    void setOgrenciNumara(std::string n) { model.setNumara(n); }
    std::string getOgrenciNumara() { return model.getNumara(); }
    // View'u güncelleme komutu
    void gorunumuGuncelle() {
        view.ogrenciDetaylariniBas(model.getIsim(), model.getNumara());
    }
};
```

4. Sistemin Çalıştırılması:

```

int main() {
    // 1. Model'i (Veriyi) oluştur
    OgrenciModel model("Ahmet Yilmaz", "2026001");
    // 2. View'u (Arayüzü) oluştur
    OgrenciView view;
    // 3. Controller'i bağla
    OgrenciController controller(model, view);
    // İlk hali ekrana bas
    controller.gorunumuGuncelle();
    // Veriyi Controller üzerinden güncelle
    controller.setOgrenciIsim("Mehmet Demir");
    // Güncel hali tekrar bas
    std::cout << "\nVeri guncellendikten sonra:\n";
    controller.gorunumuGuncelle();
}

/* Program Çıktısı:
--- Ogrenci Bilgileri ---
Ad: Ahmet Yilmaz
No: 2026001
-----

Veri guncellendikten sonra:
--- Ogrenci Bilgileri ---
Ad: Mehmet Demir
No: 2026001
-----
*/

```

Buradaki mimari ile ilgili şunlar söylenebilir;

- ♦ **Bağımsızlık:** Dikkat edersen `OgrenciModel` sınıfının içinde ne bir `cout` var ne de `OgrenciView` referansı. Bu sayede yarın bir gün veriyi ekrana değil de bir dosyaya yazmak istersen (yeni bir `View`), Model koduna dokunman gerekmez.
- ♦ **Bellek Yönetimi:** Örnekte referans (&) kullandım. Ancak büyük projelerde nesne ömürlerini yönetmek için `std::shared_ptr` veya `std::unique_ptr` kullanmak daha sağlıklı olacaktır.
- ♦ **Gözlemci Tasarım Deseni İlişkisi:** Gelişmiş MVC yapılarında, `Model` değiştiğinde `Controller`'a haber vermesi için genellikle **gözlemci tasarım deseni** ile birlikte kullanılır.

17.6 Düşün ve Çöz: Tasarım Desenleri Antrenmanı

Bu bölümdeki sorular, desenlerin sadece "nasıl" yazıldığını değil, "neden" ve "nerede" kullanılması gerektiğini sorgulamanız için tasarlanmıştır.

§17.6.1 İmalat İle İlgili (Creational) Desenler

1. **Singleton & Thread Safety:** Bir `Singleton` sınıfının `getInstance()` metodunda `static` yerel değişken kullanmak (Meyers' Singleton) neden C++11 sonrası için güvenli kabul edilir?
2. **Factory Method vs. Abstract Factory:** Bir oyun motorunda sadece "Düşman" üretmek istiyorsanız hangi deseni, "Düşman + Düşmanın Silahı + Düşmanın Zırhı"ndan oluşan uyumlu bir set üretmek istiyorsanız hangi deseni seçersiniz? Neden?
3. **Builder & Immutability:** Bir nesne inşa edildikten sonra durumunun (`state`) değiştirilmesini istemiyorsanız, `Builder` deseni size nasıl bir avantaj sağlar?
4. **Prototype & Performance:** Veritabanından 50 farklı tabloyu join ederek oluşturulan ağır bir nesneyi 1000 kez kopyalamanız gerekiyorsa, neden `new` yerine `clone()` tercih edilmelidir?
5. **Dependency Injection:** Bir sınıfın içinde başka bir sınıfı `new` ile oluşturmak yerine, onu kurucu fonksiyon üzerinden dışarıdan almanın "Test Edilebilirlik" açısından önemi nedir?

§17.6.2 Yapısal (Structural) Desenler

6. **Adapter & Legacy Code:** Eski bir kütüphanedeki `draw_circle(int, int, int)` fonksiyonunu, yeni sisteminizdeki `IDrawable` arayüzüne nasıl entegre edersiniz?
7. **Bridge & Class Explosion:** 5 farklı işletim sistemi ve 3 farklı dosya formatı (PDF, XML, JSON) için çıktı veren bir sistemde Bridge kullanılmazsa toplam kaç sınıf oluşur? Bridge ile bu sayı kaçta düşer?
8. **Decorator & Composition:** Bir nesneye çalışma zamanında (runtime) özellik eklerken neden kalıtım yerine sarmalama (`wrapping`) tercih edilir?

9. **Facade & Clean Code:** Bir kütüphanenin 20 farklı sınıfını kullanmak zorunda olan bir istemciye (client) neden doğrudan bu sınıfları değil de bir **Facade** sınıfını sunmalısınız?
10. **Proxy & Security:** Bir nesneye erişmeden önce kullanıcının yetkisini kontrol etmek istiyorsanız, bu mantığı asıl nesnenin içine mi yoksa bir **Proxy** içine mi yazmalısınız? Neden?
11. **Flyweight & RAM:** Bir metin düzenleyicideki her bir karakter için ayrı birer nesne oluşturmak belleği nasıl etkiler? Paylaşılan (intrinsic) veri ne olmalıdır?

§17.6.3 Davranış (Behavioral) Desenleri

12. **Chain of Responsibility:** Bir HTTP isteğinin önce "Güvenlik", sonra "Loglama", en son "Veri İşleme" aşamalarından geçmesi gerekiyorsa, zincirin sırasını çalışma zamanında değiştirmek bize ne kazandırır?
13. **Command & Undo:** Bir "Geri Al" (**undo**) mekanizması kurarken neden komutun sadece kendisini değil, operasyon öncesindeki durumu da saklaması gerekir?
14. **Iterator & Encapsulation:** Bir **List** nesnesinin içindeki **std::vector** yapısını **std::list** olarak değiştirirsek, **Iterator** kullanan istemci kodları bundan etkilenir mi?
15. **Mediator & Air Traffic:** 10 uçağın birbirinin konumunu bilmesi yerine neden sadece kulenin (**Mediator**) tüm uçakları bilmesi daha güvenlidir? Nesneler arasındaki bağımlılık (**coupling**) nasıl değişir?
16. **Observer & Memory Leaks:** Bir nesne (**Subject**) silindiğinde, ona abone olan (**Observer**) nesnelerin listeden çıkarılmaması ne gibi sorunlara yol açar?
17. **State Pattern:** Bir ATM makinesinin durumlarını yönetirken devasa bir **switch-case** bloğu yerine **State** desenini kullanmanın en büyük avantajı nedir?
18. **Strategy & Open-Closed:** Bir sıralama algoritmasını çalışma zamanında değiştirmek istediğinizde **Strategy** deseni "Open-Closed" prensibini nasıl sağlar?
19. **Template Method vs. Strategy:** Algoritmanın iskeletini sabit tutup adımlarını alt sınıfa bırakmakla, algoritmanın tamamını bir nesne olarak değiştirmek arasındaki temel fark nedir?
20. **Visitor & Double Dispatch:** Bir sınıf hiyerarşisine yeni bir işlem eklemek için neden sınıfların kodunu değiştirmek yerine **Visitor** kullanmalıyız?